

Computer Organisation

Topic 3

Computer Arithmetic

Outline

1

Arithmetic & Logic
Unit

2

Integer
Representation

- Sign-magnitude
- Twos complement
- Conversion
between lengths

3

Integer Arithmetic

- Addition &
Subtraction
- Multiplication
- Division

4

Floating-point
Arithmetic

Learning objectives



1010
1010

Understand the distinction between the way in which numbers are represented (the binary format) and the algorithms used for the basic arithmetic operations.



Explain twos complement representation.

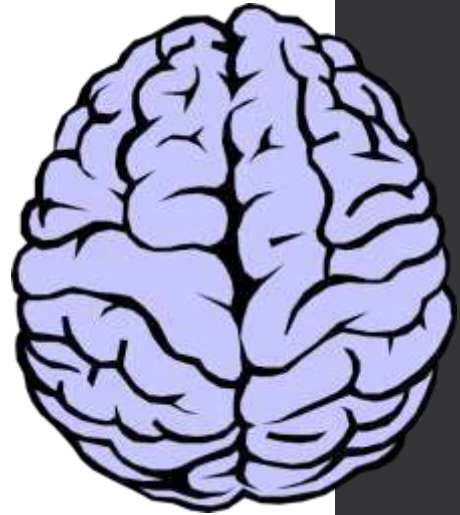


Present an overview of the techniques for doing basic arithmetic operations in twos complement notation.



Understand the use of significand, base, and exponent in the representation of floating-point numbers.

Arithmetic & Logic Unit

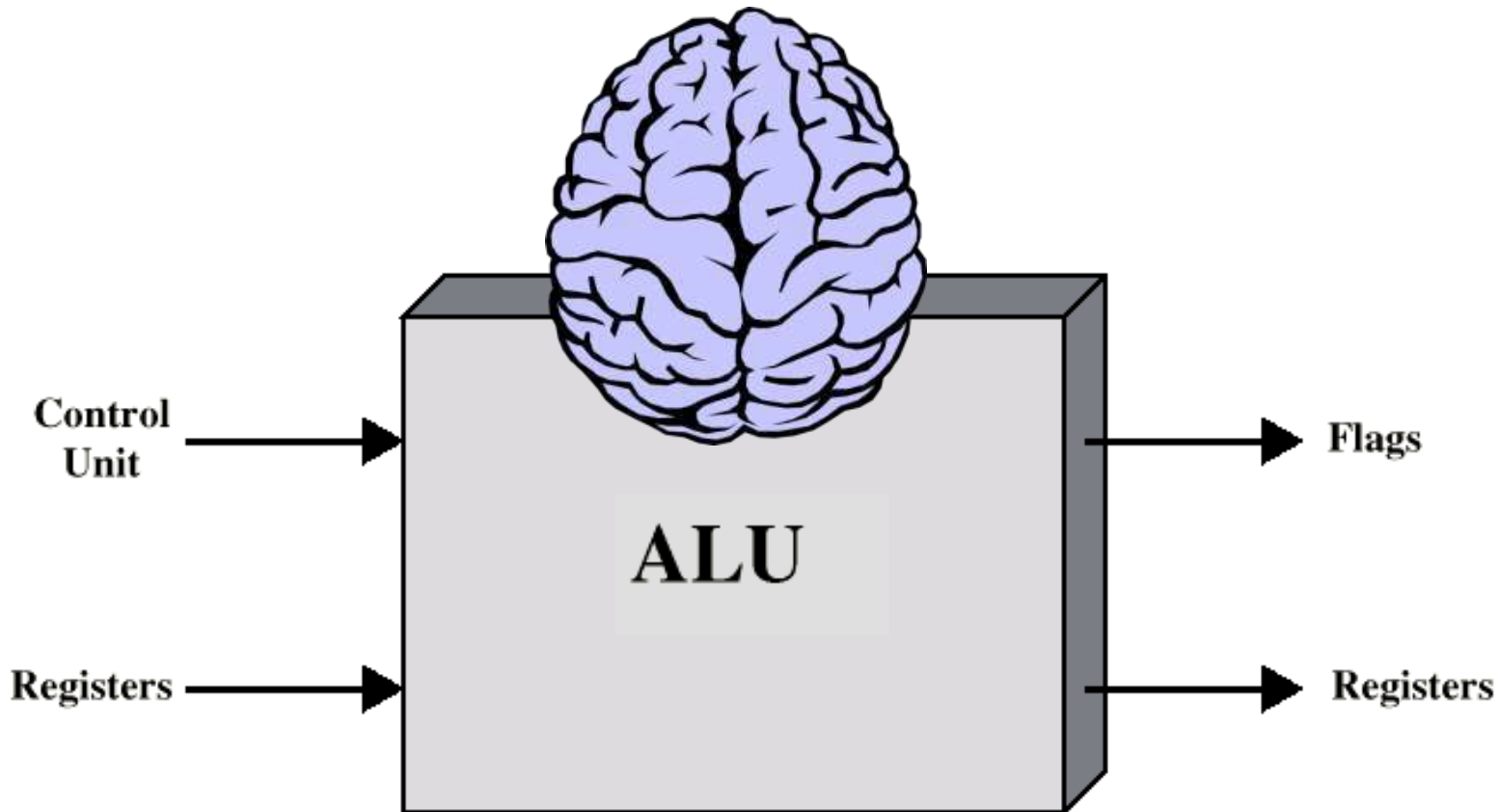


- A vital component of the CPU responsible for executing arithmetic and logical instructions
- A fundamental part of a computer's processing capabilities
- Other elements of the computer system (CU, registers, memory) mainly to bring data into the ALU for it to process and then to take the results back out
- Does the **calculations**
- Everything else in the computer is there to service this unit

Arithmetic & Logic Unit

- Handles **integers**
- May handle **floating point** (real) numbers

ALU Inputs and Outputs



Outline

1

Arithmetic & Logic
Unit

2

Integer
Representation

- Sign-magnitude
- Twos complement
- Conversion
between lengths

3

Integer Arithmetic

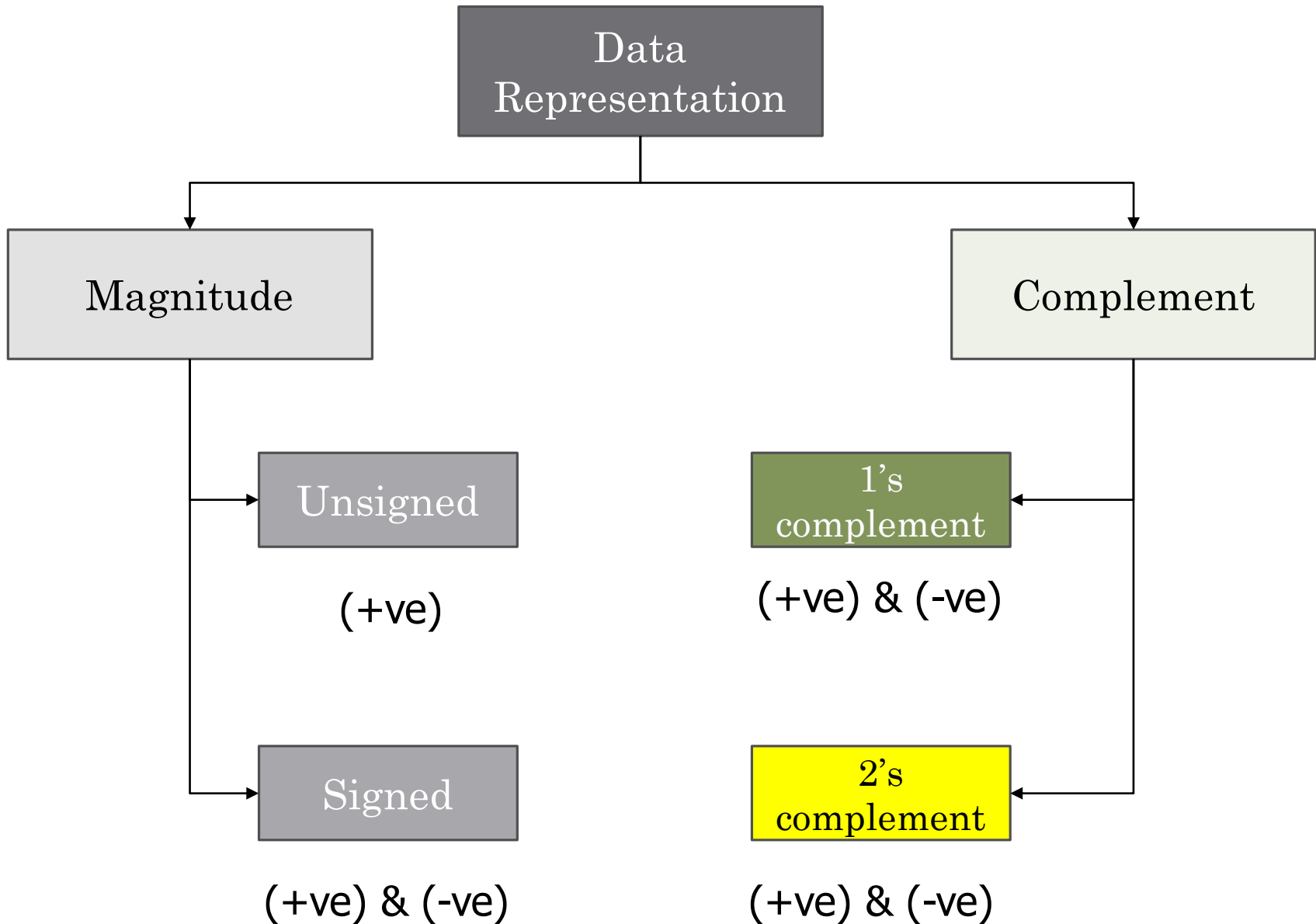
- Addition &
Subtraction
- Multiplication
- Division

4

Floating-point
Arithmetic

Integer Representation

- The way in which whole numbers (integers) are represented in a digital computer/binary system
- Only have **0** & **1** to represent everything
- Positive numbers stored in binary
 - e.g. 41=00101001
- Binary digit or a **Bit**



In all representation, (+) number is represented in the same way

Integer Representation

- In the binary system, number system can be represented with just the digits 0 and 1

An 8-bit word can represent the numbers from 0 to 255, such as

$$00000000 = 0$$

$$00000001 = 1$$

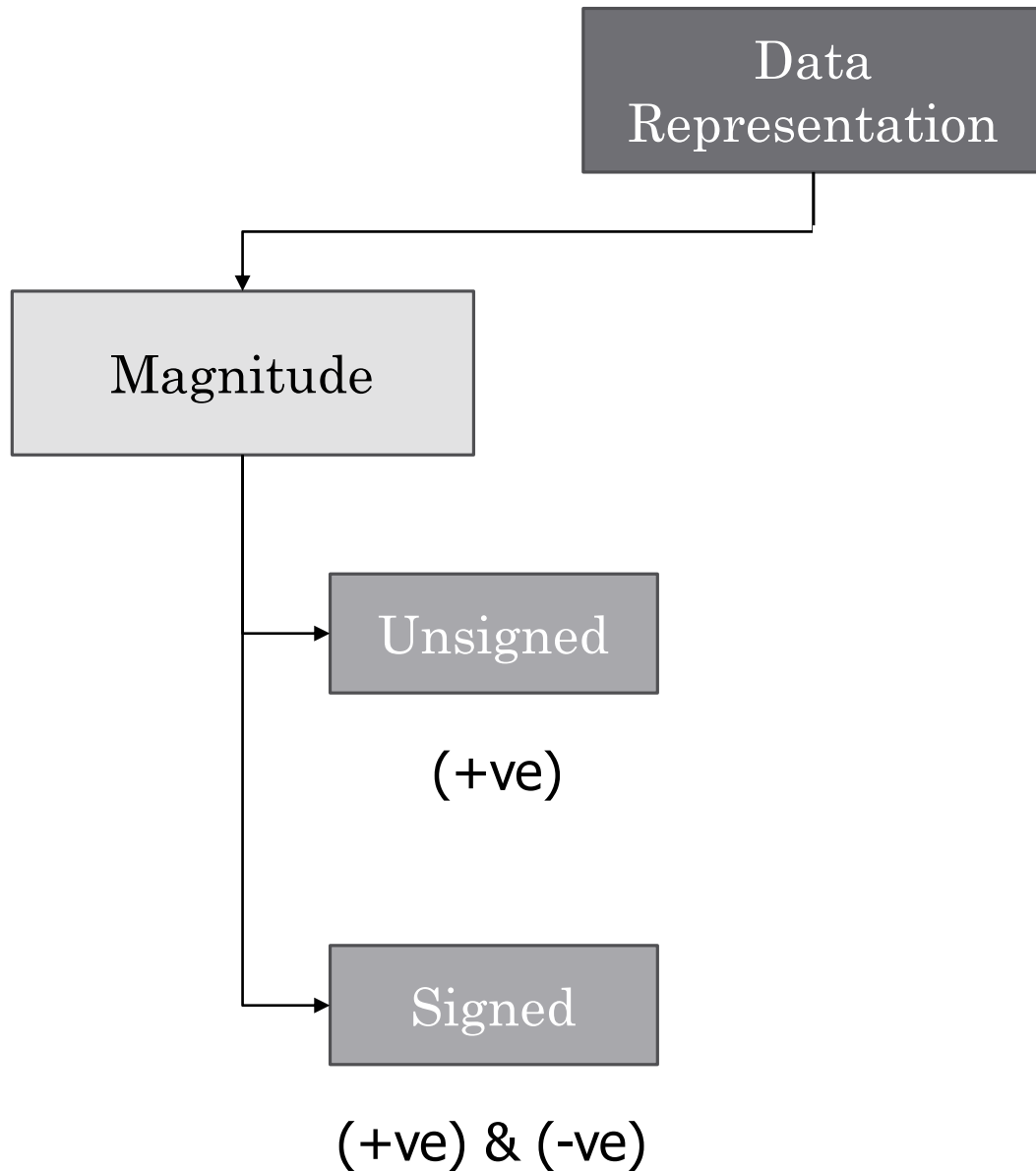
$$00101001 = 41$$

$$10000000 = 128$$

$$11111111 = 255$$

Magnitude

Two's complement



In all representation, (+) number is represented in the same way

Unsigned magnitude

- Represents **positive integers**

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Pattern	1	0	0	1	1	1	0	1
Weight	128	64	32	16	8	4	2	1

10011101 = unsigned representation of 157

Unsigned magnitude

- Only accept positive numbers
- In computer programming, the unsigned integers property open for application in most numeric data types:
 - Short
 - Long
 - Int
 - Char

Advantages and disadvantages of unsigned magnitude

Advantages

- One representation of zero
- Simple addition

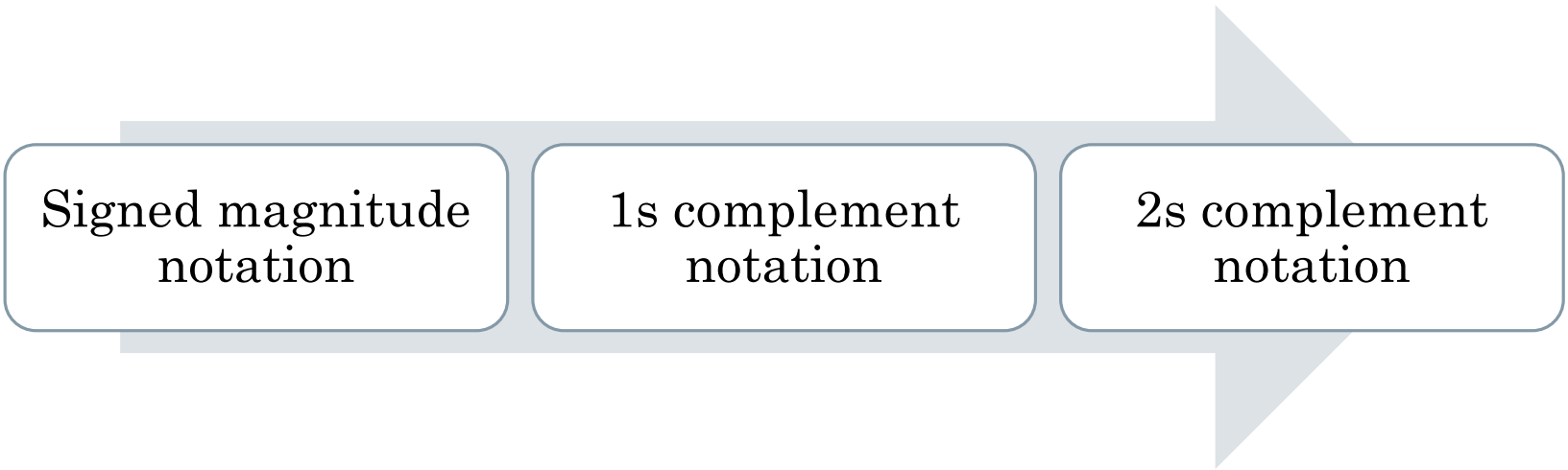
Disadvantages

- Negative numbers cannot be represented
- The need of different notation to represent negative numbers

Is a representation of negative numbers possible?

Unfortunately,

- You **cannot** just stick a negative sign in front of a binary number (it does not work like that)
- There are three methods used to represent negative numbers:



Signed magnitude
notation

1s complement
notation

2s complement
notation

Signed-magnitude

- Human use the signed-magnitude system.
- We add + or – to the front of a number to indicate its sign
- We can do this in binary too! By adding a sign bit in front of our numbers.

Signed-Magnitude

- Use **one-bit** to represent the sign of the integer
- Left most bit (Most Significant Bit, MSB) is sign bit

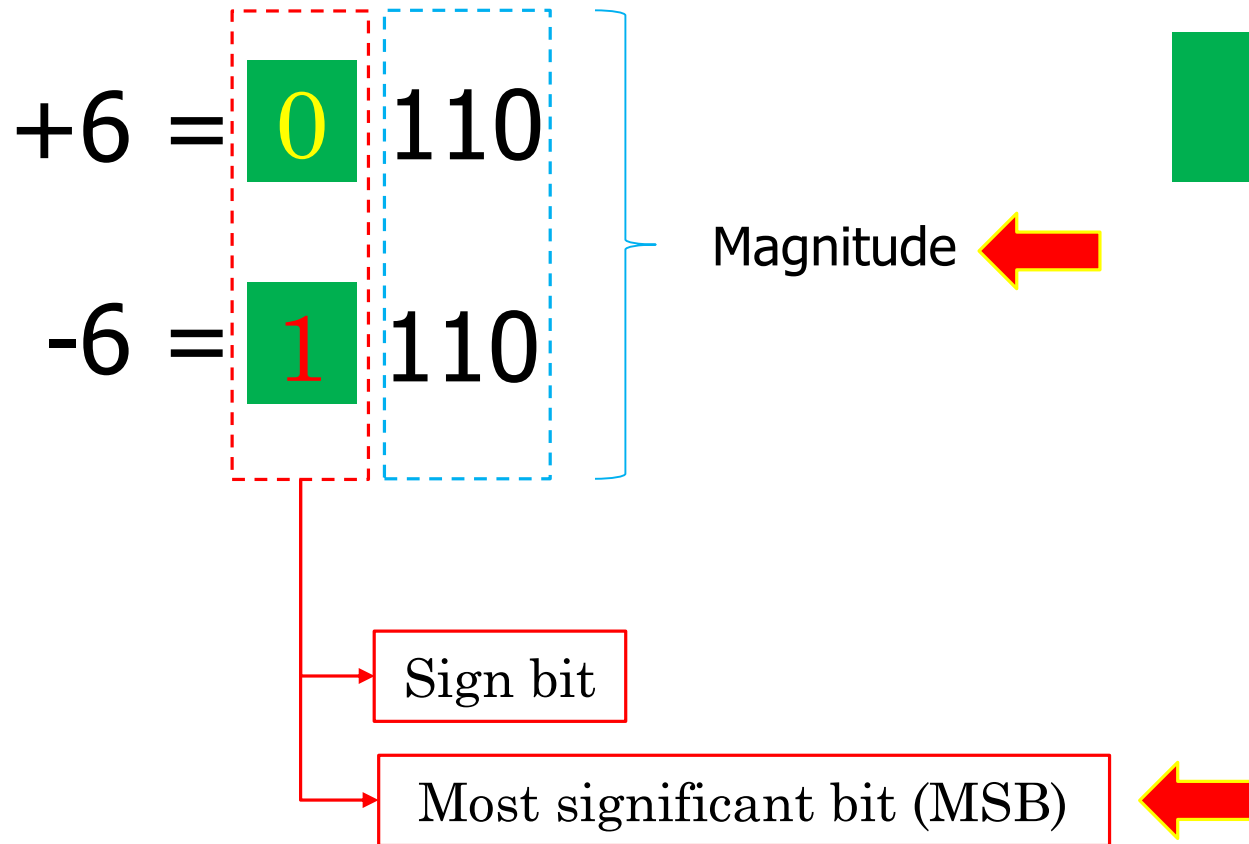
0 means **positive**

+18 = **0**0010010

1 means **negative**

-18 = **1**0010010

Signed-Magnitude



Signed-Magnitude

0 = (+)

1 = (-)

$$+13 = 0 \ 1101$$

$$-13 = 1 \ 1101$$

Signed-magnitude

- Suppose 10011101 is a signed magnitude representation.
- The sign bit is 1, then the number represented is negative

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Pattern	1	0	0	1	1	1	0	1
Weight				16	8	4		1

- The magnitude is 0011101 with a value of $2^4 + 2^3 + 2^2 + 2^0 = 29$
- Then, the number represented by **10011101** is **-29**.

Signed-Magnitude Range

- The range of signed magnitude representation depends on the number of bits used for both the sign bit and the magnitude part
- Sign Bit part: 1 bit (1 for negative, 0 for positive)
- Magnitude Part: (n-1) bits (assuming 1 bit is used for the sign)
- The range of values that can be represented in signed magnitude representation with n bits is typically:

Range of signed magnitude = - $(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$

n = number of bit

Signed-Magnitude Range

n = number of bit

$$-(2^{n-1} - 1) \text{ to } +(2^{n-1} - 1)$$

Example:

$$n = 4$$

$$-(2^{4-1} - 1) \text{ to } +(2^{4-1} - 1)$$

$$-(2^3 - 1) \text{ to } +(2^3 - 1)$$

$$-(2^3 - 1) \text{ to } +(2^3 - 1)$$

$$-(8 - 1) \text{ to } +(8 - 1)$$

Answer:

The signed magnitude for 4 bit are between:

$$-(7) \text{ to } +(7)$$

Drawbacks of sign-magnitude

- Addition and subtraction require a consideration of both the signs of the numbers and their relative magnitudes to carry out the required operation.
- They are two representations of 0

$$00000000 = +0_{10}$$

$$10000000 = -0_{10}$$

- To test if a number is 0 or not, the CPU will need to see whether it is 00000000 or 10000000.
- Therefore, having 2 representation of 0 is inconvenient.

Quick Activity #1

- a. 37_{10} has 00100101 is signed magnitude notation.
Find the signed magnitude of -37_{10} .
- b. Using the signed magnitude notation find the 8-bit binary representation of the decimal value 24_{10} and -24_{10} .
- c. Find the signed magnitude of -63 using 8-bit binary sequence?

Quick Activity #1

① $37_{10} = 00100101$ (+ve)

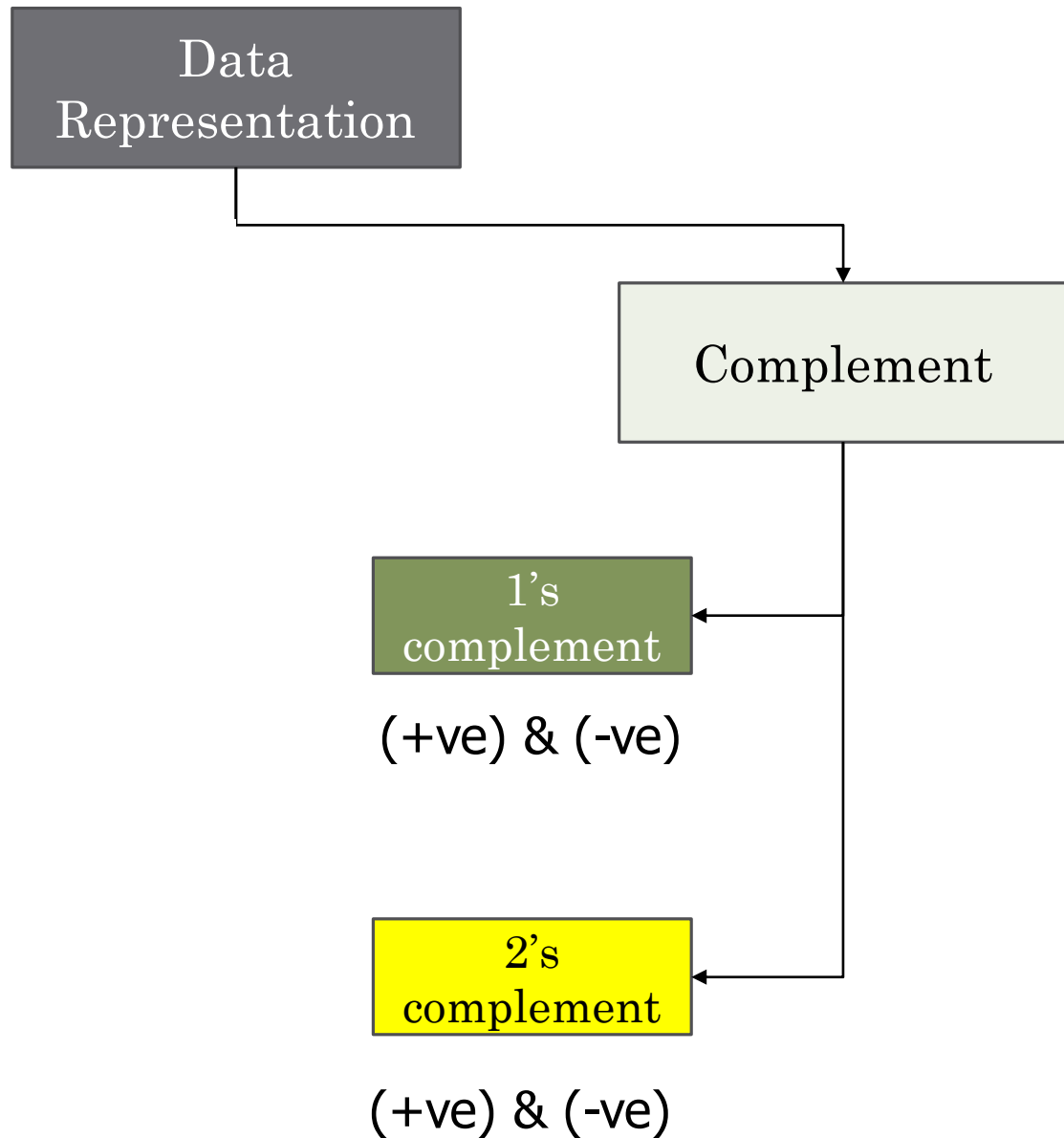
$-37_{10} = 1010010$

② 8-bit representation of 24_{10} & -24_{10}

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
<u>0</u>	0	0	1	1	0	0	0	$= 24_{10}$
<u>1</u>	0	0	1	1	0	0	0	$= -24_{10}$

③ 8-bit of $-63_{10} = 10111111_2$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
1	0	1	1	1	1	1	1	



In all representation, (+) number is represented in the same way

1s complement

- In a different representation, 1s complement, we negate numbers by complementing each bit of the number.
- We keep the sign bits; 0 for positive and 1 for negative
- The sign bit is complemented along with the rest of the bits

1's complement

- Negative numbers are represented by taking the bitwise complement (**inverting all the bits**) of the positive binary representation of the number

$$(1010)_2 = (0101)_2$$

2's complement

- 2's complement is a binary representation method used in computers to represent signed integers (both positive and negative numbers).
- It is the most common method for representing signed integers
- 2's complement requires 2 rules:

Rule
1

Find 1's complement for the integer

Rule
2

Add 1 to the 1's complement

Step 1: Find 1's complement for the integer

Step 2: Add 1 to the 1's complement

2's complement

Example:

$(10111010)_2$

Step 1

01000101

Step 2

$$\begin{array}{r} \text{Step 2} \quad 01000101 \\ \quad \quad \quad + \quad \quad \quad 1 \\ \hline 01000110 \\ \hline \end{array}$$

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 carry forward 1

2's complement (Alt)

1

Most significant bit (MSB) is **sign**
bit (Left)

2

One representation of **zero**

= 1's complement

3

Add 1 to LSB

(Right)

+3	=	00000011
+2	=	00000010
+1	=	00000001
+0	=	00000000
-1	=	11111111
-2	=	11111110
-3	=	11111101

2's complement

1

+3 = 00000011

2

1's complement
11111100

3

Add 1 to LSB

11111101

+3 = 00000011

+2 = 00000010

+1 = 00000001

+0 = 00000000

-1 = 11111111

-2 = 11111110

-3 = 11111101

2's complement (Alt)

1

Identify the LSB and copy all zeros until the first 1

2

Copy the first 1

3

Complement all the remaining bits (after 1)

$$+3 = 00000011$$

$$+2 = 00000010$$

$$+1 = 00000001$$

$$+0 = 00000000$$

$$-1 = 11111111$$

$$-2 = 11111110$$

$$-3 = 11111101$$

2's complement

1

$$+3 = 00000011$$

2

11111101

$$+3 = 00000011$$

$$+2 = 00000010$$

$$+1 = 00000001$$

$$+0 = 00000000$$

$$-1 = 11111111$$

$$-2 = 11111110$$

$$-3 = 11111101$$

Quick Activity #2

- 01001_2
- 01011_2
- 00111_2
- 00001_2

Characteristics of 2's Complement Representation

Range	-2^{n-1} through $2^{n-1} - 1$
Number of Representations of Zero	One
Negation	Take the Boolean complement of each bit of the corresponding positive number, then add 1 to the resulting bit pattern viewed as an unsigned integer.
Expansion of Bit Length	Add additional bit positions to the left and fill in with the value of the original sign bit.
Overflow Rule	If two numbers with the same sign (both positive or both negative) are added, then overflow occurs if and only if the result has the opposite sign.
Subtraction Rule	To subtract B from A , take the two's complement of B and add it to A .

n = number of bit

$- (2^{n-1})$ to $+(2^{n-1} - 1)$

Advantages of 2's complement

- It eliminates the need for two representations of zero, resulting in a unique representation.
- Arithmetic operations like addition, subtraction, and multiplication are simplified, and there are no issues with end-around carries.
- The representation is symmetric, meaning the range of positive and negative values is evenly distributed around zero.
- It's widely supported and used in modern computer systems due to its efficiency in arithmetic operations and other benefits.

Advantages of 2's complement

Subtraction can
be easily
performed

Multiplication
is just a
repeated
addition

Division is just
a repeated
subtraction

2s complement
is widely used
in ALU

Converting Between 2's Complement Binary To Decimal

Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Weight	128	64	32	16	8	4	2	1
Bit Pattern								

An eight-position 2's complement value box

Converting Between 2's Complement Binary To Decimal

Example:

Convert binary 10000011 to decimal

128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

-128

+2

+1

$$10000011 = -128 + 2 + 1$$

$$10000011 = -128 + 3$$

10000011 = -125

Converting Between 2's Complement Decimal To Binary

Example:

Convert decimal 120 to binary

128	64	32	16	8	4	2	1
0	1	1	1	1	0	0	0
	+64	+32	+16	+8			

$$64 + 32 + 16 + 8 = 01111000$$

$$120 = 01111000$$

Converting Between 2's Complement Decimal To Binary

Example:

Convert decimal -120 to binary

-128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

-128 +8

$$-128 + 8 = 10001000$$

-120 = 10001000

In-class activity

Convert the following:

10100101

66

01101101

-80

128	64	32	16	8	4	2	1

Negation

- In signed-magnitude representation, the rule for forming the negation of an integer is simple: invert the sign bit.
- In twos complement notation, the negation of an integer can be formed with the following rules:
 1. Find 1's complement to the integer
 2. Add 1 to 1's complement integer

$$\begin{aligned}
 +18 &= 00010010 \quad (\text{twos complement}) \\
 \text{bitwise complement} &= 11101101 \\
 &\quad + \quad \underline{1} \\
 &\quad 11101110 = -18
 \end{aligned}$$

$$\begin{aligned}
 -18 &= 11101110 \quad (\text{twos complement}) \\
 \text{bitwise complement} &= 00010001 \\
 &\quad + \quad \underline{1} \\
 &\quad 00010010 = +18
 \end{aligned}$$

$$\begin{aligned}
 0 &= 00000000 \quad (\text{twos complement}) \\
 \text{bitwise complement} &= 11111111 \\
 &\quad + \quad \underline{1} \\
 &\quad 100000000 = 0
 \end{aligned}$$

$$\begin{aligned}
 +128 &= 10000000 \quad (\text{twos complement}) \\
 \text{bitwise complement} &= 01111111 \\
 &\quad + \quad \underline{1} \\
 &\quad 10000000 = -128
 \end{aligned}$$

Quick Activity #3

Represent these numbers in 8-bit 2's complement

- a. + 44
- b. + 89
- c. + 106
- d. - 18
- e. - 74
- f. - 86
- g. - 99
- h. - 110
- i. - 120
- j. - 128

2's complement operations

Addition

Subtraction

Multiplication

Division

Addition in 2's in complement

$\begin{array}{r} 1001 = -7 \\ + \underline{0101} = 5 \\ \hline 1110 = -2 \end{array}$ (a) $(-7) + (+5)$	$\begin{array}{r} 1100 = -4 \\ + \underline{0100} = 4 \\ \hline 10000 = 0 \end{array}$ (b) $(-4) + (+4)$
$\begin{array}{r} 0011 = 3 \\ + \underline{0100} = 4 \\ \hline 0111 = 7 \end{array}$ (c) $(+3) + (+4)$	$\begin{array}{r} 1100 = -4 \\ + \underline{1111} = -1 \\ \hline 11011 = -5 \end{array}$ (d) $(-4) + (-1)$
$\begin{array}{r} 0101 = 5 \\ + \underline{0100} = 4 \\ \hline 1001 = \text{Overflow} \end{array}$ (e) $(+5) + (+4)$	$\begin{array}{r} 1001 = -7 \\ + \underline{1010} = -6 \\ \hline 10011 = \text{Overflow} \end{array}$ (f) $(-7) + (-6)$

Addition and Subtraction

- Addition proceeds as if the two numbers were **unsigned integers**
- If the result of the operation is positive, we get a positive number in 2's complement form, which is the same as in unsigned integer form
- If the result of the operation is negative, we get a negative number in 2's complement form
- In some instances, there is a carry bit beyond the end of the word (indicated by shading), which is ignored
- On any addition, the result may be larger than can be held in the word size being used = **OVERFLOW**
- When overflow occurs, the ALU must signal this fact so that no attempt is made to use the result

$$\begin{array}{r} 1001 = -7 \\ + \underline{0101} = 5 \\ 1110 = -2 \end{array}$$

(a) $(-7) + (+5)$

$$\begin{array}{r} 1100 = -4 \\ + \underline{0100} = 4 \\ \textcolor{teal}{1}0000 = 0 \end{array}$$

(b) $(-4) + (+4)$

$$\begin{array}{r} 0011 = 3 \\ + \underline{0100} = 4 \\ 0111 = 7 \end{array}$$

(c) $(+3) + (+4)$

$$\begin{array}{r} 1100 = -4 \\ + \underline{1111} = -1 \\ \textcolor{teal}{1}1011 = -5 \end{array}$$

(d) $(-4) + (-1)$

$$\begin{array}{r} 0101 = 5 \\ + \underline{0100} = 4 \\ 1001 = \text{Overflow} \end{array}$$

(e) $(+5) + (+4)$

$$\begin{array}{r} 1001 = -7 \\ + \underline{1010} = -6 \\ \textcolor{teal}{1}0011 = \text{Overflow} \end{array}$$

(f) $(-7) + (-6)$

Rules of Binary Arithmetic

Binary Addition

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$ Sum of 0 with a carry of 1

Binary Subtraction

- $0 - 0 = 0$
- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ (Borrow 1)

Binary Multiplication

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

Addition (Example 1)

Add two 4-bits 2's complement of 4 and 3

Step 1 – Convert 4 and 3 to binary

	128	64	32	16	8	4	2	1
4					0	1	0	0
3					0	0	1	1

Step 2 – Perform binary addition

4					0	1	0	0
3					0	0	1	1
					0	1	1	1

Answer: $4 + 3 = 7 = 0111$

Addition (Example 2)

Add two 5-bit 2's complement of 13 and -6

Step 1 – Convert 13 and -6 to binary

	128	64	32	16	8	4	2	1
13				0	1	1	0	1

	128	64	32	16	8	4	2	1
+6				0	0	1	1	0
-6				1	1	0	1	0

In 2s
complement

Step 2 – Perform binary addition

13			0	1	1	0	1
-6			1	1	0	1	0
		1	0	0	1	1	1

Answer: $13 + (-6) = 7 = 00111$

Addition (Example 3)

Add two 5-bit 2's complement of -15 and 9

Step 1 – Convert -15 and 9 to binary

	128	64	32	16	8	4	2	1
+15				0	1	1	1	1
-15				1	0	0	0	1

In 2s
complement

	128	64	32	16	8	4	2	1
9				0	1	0	0	1

Step 2 – Perform binary addition

15			1	0	0	0	1
-6			0	1	0	0	1
			1	1	0	1	0

Answer: $-15 + 9 = -6 = 11010$

Addition (Example 4)

Add two 8-bit 2's complement of -39 and 92

Step 1 – Convert -39 and 92 to binary

	128	64	32	16	8	4	2	1
+39	0	0	1	0	0	1	1	1
-39	1	1	0	1	1	0	0	1

In 2s
complement

	128	64	32	16	8	4	2	1
92	0	1	0	1	1	1	0	0

Step 2 – Perform binary addition

-39	1	1	0	1	1	0	0	1
92	0	1	0	1	1	1	0	0
1	0	0	1	1	0	1	0	1

Answer: $-39 + 92 = 53 = 00110101$

Addition (Example 5)

Add two 5-bit 2's complement of -8 and -4

Step 1 – Convert -8 and -4 to binary

128	64	32	16	8	4	2	1
+8			0	1	0	0	0
-8			1	1	0	0	0
+4			0	0	1	0	0
-4			1	1	1	0	0

Step 2 – Perform binary addition

-8			1	1	0	0	0
-4			1	1	1	0	0
		1	1	0	1	0	0

Answer: $-8 + (-4) = -12 = 10100$

Addition (Example 4)

**Overflow
example**

Add two 8-bit 2's complement of $104 + 45$

Step 1 – Convert 104 and 45 to binary

	128	64	32	16	8	4	2	1
104	0	1	1	0	1	0	0	0
45	0	0	1	0	1	1	0	1

Step 2 – Perform binary addition

104	0	1	1	0	1	0	0	0
45	0	0	1	0	1	1	0	1
	1	0	0	1	0	1	0	1

Answer: $104 + 45 = 149 = 10010101$

How do we handle overflow?

- Since we added two positive numbers and got a negative result, overflow has indeed occurred. Therefore, this answer is not correct for 8-bit representation.
- So, the correct answer cannot be represented within an 8-bit signed integer and would **require a larger bit width** (like 9 bits or more) to hold the value accurately.

Quick Activity #4

Assume numbers are represented in 8-bit two's-complement representation. Show the calculation of the following:

1. $-19 + (-7)$

2. $44 + 45$

3. $-75 + 59$

4. $127 + 1$

5. $-1+1$

6. $-103 + (-69)$

Decimal Representation	Sign-Magnitude Representation	Twos Complement Representation	Biased Representation
+8	—	—	1111
+7	0111	0111	1110
+6	0110	0110	1101
+5	0101	0101	1100
+4	0100	0100	1011
+3	0011	0011	1010
+2	0010	0010	1001
+1	0001	0001	1000
+0	0000	0000	0111
−0	1000	—	—
−1	1001	1111	0110
−2	1010	1110	0101
−3	1011	1101	0100
−4	1100	1100	0011
−5	1101	1011	0010
−6	1110	1010	0001
−7	1111	1001	0000
−8	—	1000	—

2's complement operations

Addition

Subtraction

Multiplication

Division

Subtraction in 2's complement

To subtract one number (subtrahend) $[S]$ from another (minuend) $[M]$, take the 2s complement (negation) of the subtrahend and add them to the minuend

$$[M] + 2s [S]$$

Step 1: Find 2s complement of subtrahend $[S]$

Step 2: Perform the addition

Step 3: If the carry bit is generated and the result is positive, and in its true form

If carry bit is not produced, then the result is negative and in its 2s complement form

Subtraction (Example 1)

Overflow
example

Subtract two 4-bit 2's complement of 2 and -7

M = 2

S = -7

Step 1 – Convert 2 and -7 to binary

M	128	64	32	16	8	4	2	1
2					0	0	1	0

S	128	64	32	16	8	4	2	1
7					0	1	1	1
-7					1	0	0	1

Step 2 – Perform binary addition

+2					0	0	1	0
+7					0	1	1	1
					1	0	0	1

Answer: $2 - (-7) = 2 + 7 = 9$

Subtraction (Example 2)

Subtract two 4-bit 2's complement of 5 and -2

M = 5

S = -2

Step 1 – Convert 5 and -2 to binary

M	128	64	32	16	8	4	2	1
5					0	1	0	1

S	128	64	32	16	8	4	2	1
2					0	0	1	0

Step 2 – Perform binary addition

5					0	1	0	1
2					0	0	1	0
					0	1	1	1

Answer: $5 - (-2) = 5 + 2 = 7 = 0111$

Answer: $5 - (-3) = 5 + 3 = 8$

Subtract two 4-bit 2's complement of 5 and -3

M = 5

S = -3

Step 1 – Convert -5 and -2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1

S	128	64	32	16	8	4	2	1
+3					0	0	1	1

Step 2 – Perform binary addition

+5					0	1	0	1
+3					0	0	1	1
				1	0	0	0	0

Subtraction (Example 4)

Subtract two 4-bit 2's complement of 5 and 2

M = 5
S = 2

Step 1 – Convert 5 and 2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1
+2					0	0	1	0
-2					1	1	1	0

Step 2 – Perform binary addition

5					0	1	0	1
-2					1	1	1	0
				1	0	0	1	1

Answer: $5 - (+2) = 5 - 2 = 3 = 0011$

Subtraction (Example 5)

Subtract two 4-bit 2's complement of 7 and 7

M = 7
S = 7

Step 1 – Convert 7 to binary

M	128	64	32	16	8	4	2	1
7					0	1	1	1
-7					1	0	0	1

Step 2 – Perform binary addition

7					0	1	1	1
-7					1	0	0	1
				1	0	0	0	0

Answer: $7 - 7 = 0 = 10000$

Subtraction (Example 6)

Overflow
example

Subtract two 4-bit 2's complement of 7 and -7

M = 7
S = -7

Step 1 – Convert 7 to binary

M	128	64	32	16	8	4	2	1
7					0	1	1	1

Step 2 – Perform binary addition

7					0	1	1	1
7					0	1	1	1
					1	1	1	0

Answer: $7 - (-7) = 14 = 1110$

Answer: $5 - (-6) = 5 + 6 = 11$

Overflow
example

Subtract two 4-bit 2's complement of 5 and -6

M = 5

S = -6

Step 1 – Convert -5 and -2 to binary

M	128	64	32	16	8	4	2	1
+5					0	1	0	1

S	128	64	32	16	8	4	2	1
+6					0	1	1	0

Step 2 – Perform binary addition

+5					0	1	0	1
+6					0	1	1	0
					1	0	1	1

Quick Activity #5

Assume numbers are represented in 8-bit twos-complement representation. Show the calculation of the following:

i. $5 - 3$

ii. $7 - 4$

iii. $-6 - 2$

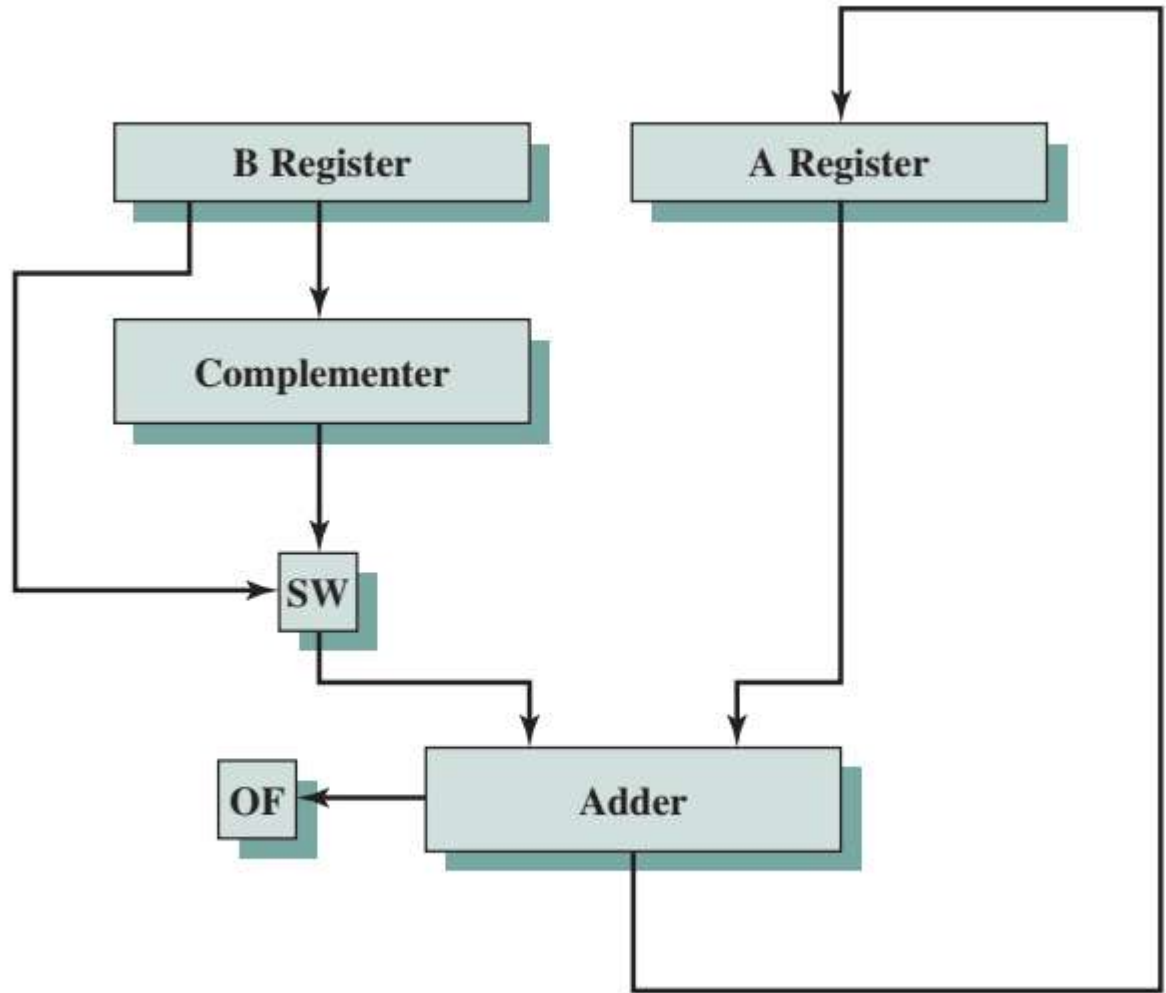
iv. $12 - (-5)$

v. $-7 - (-8)$

vi. $45 - 18$

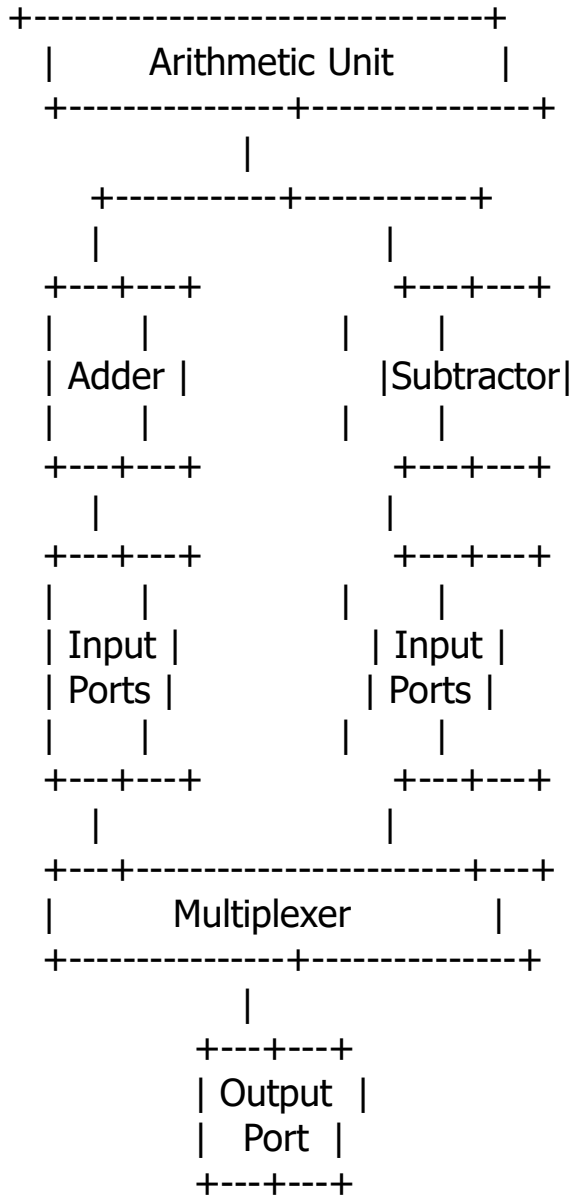
vii. $-50 - 25$

Hardware for Addition and Subtraction



OF = Overflow bit

SW = Switch (select addition or subtraction)



1. The control signals for the multiplexer determine whether the arithmetic unit performs addition or subtraction.
2. The output of the selected operation is then passed to the output port.
3. This basic hardware setup can be expanded and optimized for more complex arithmetic operations and can be part of a larger CPU or digital processing system.

Computer Organisation

Topic 3

Computer Arithmetic

(Part 2)

Outline

1 Arithmetic & Logic Unit

2 Integer Representation

- Sign-magnitude
- Twos complement
- Conversion between lengths

3 Integer Arithmetic

- Addition & Subtraction
- Multiplication
- Division

4 Floating-point Arithmetic

2's complement operations

Addition

Subtraction

Multiplication

Division

Add-shift method (unsigned integer)
Booth's algorithm (-ve numbers)

Multiplication

- Compared with addition and subtraction, multiplication is a **complex operation**, whether performed in hardware or software.
- A wide variety of algorithms have been used in various computers.
- Work out **partial product** for each digit
- Take care with place value (column)
- **Add partial products**

Multiplication of unsigned integer

1011		Multiplicand (11)
×1101		Multiplier (13)
1011	}	
0000		
1011		Partial products
1011		
10001111		Product (143)

11 x 13

- 11 = multiplicand
- 13 = multiplier

				1	0	1	1	11				
				X	1	1	0	1	13			
Partial products	{				1	0	1	1	{	Addition operation		
				0	0	0	0	x				
				1	0	1	1	x			x	
				1	0	1	1	x			x	x
				1	0	0	0	1	1	1	1	

Note:

- if multiplier bit is 1 copy multiplicand (place value) otherwise zero
- Need double length result

Multiplication

- How can this be more efficient?
 1. Perform running addition on partial products
 - Do not have to wait until the end
 - Fewer registers are needed

This eliminates the need for storage of all the partial products

Multiplication

- How can this be more efficient?

2. Save time on generation of partial products

- i. For each **1** on multiplier requires **an add** and **a shift** operation
- ii. For each **0** on multiplier, only **a shift** is required

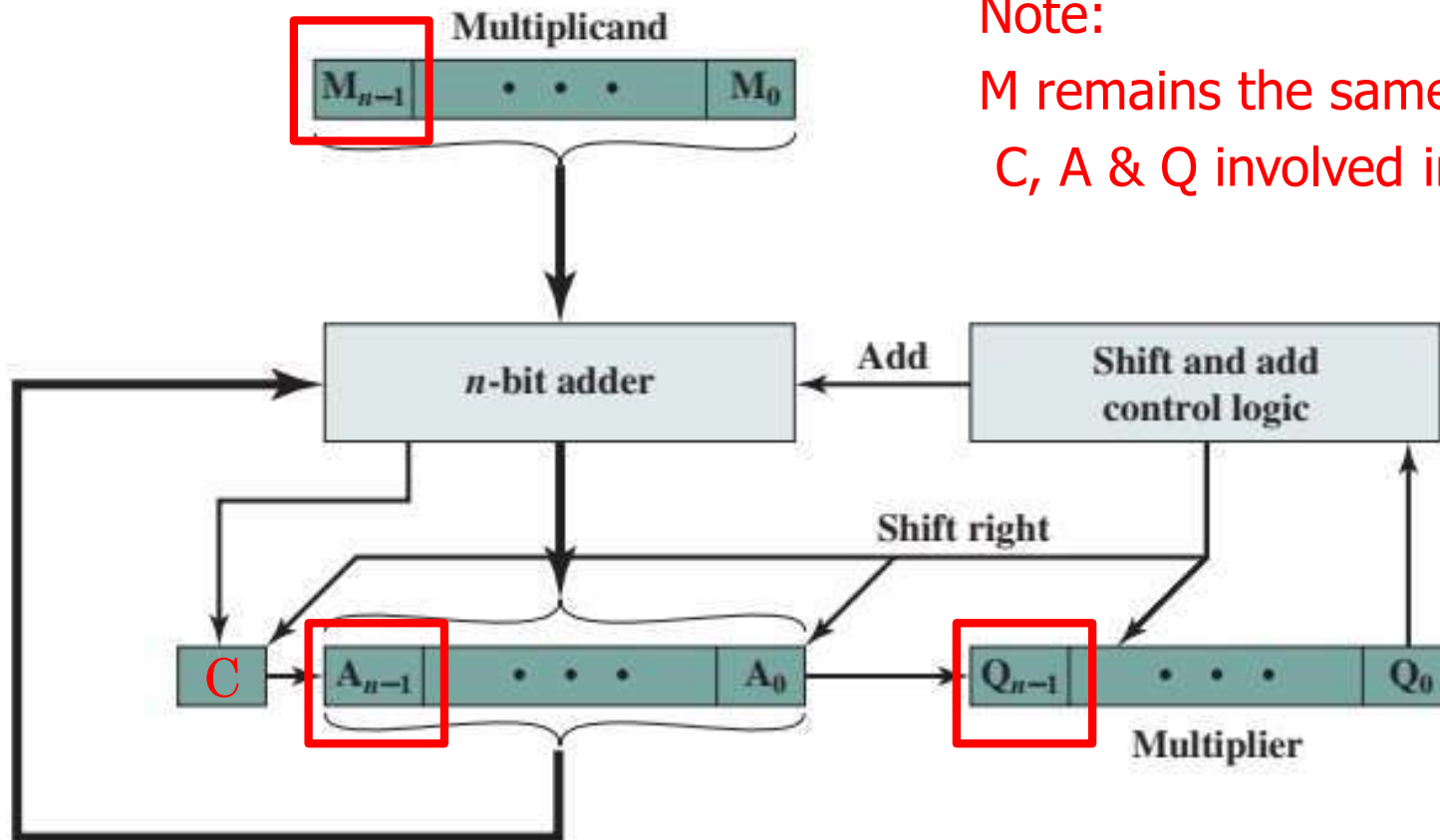
M: $(11)_{10}$

Q: $(13)_{10}$

A: (additional register)

C: Carry bit

Multiplication block diagram



Note:

M remains the same

C, A & Q involved in shifting

For each **1** on multiplier requires **an add** and **a shift** operation
 For each **0** on multiplier, only **a shift** is required

Unsigned Binary Multiplication

M: Multiplicand = $(11)_{10}$

Q: Multiplier = $(13)_{10}$

A: (additional register)

C: Carry bit

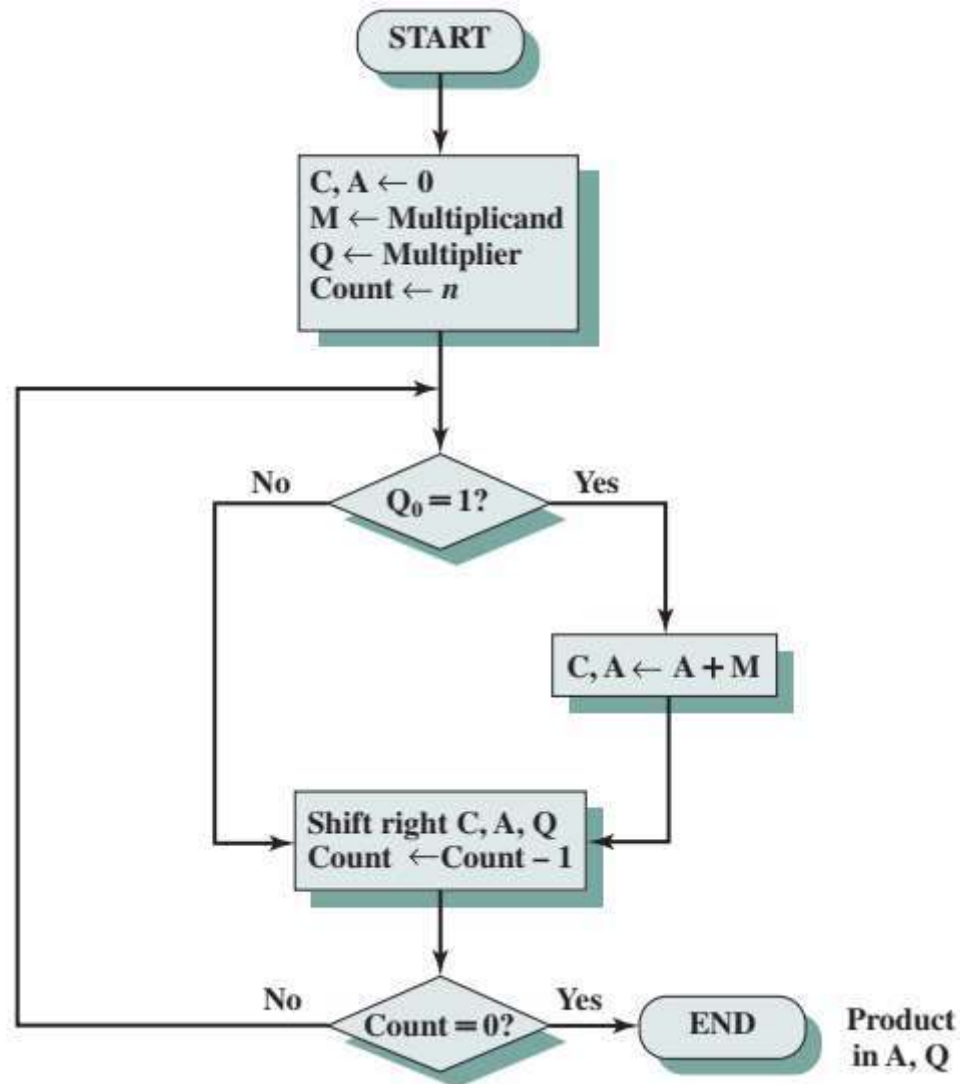
C	A	Q	M	Initial values	
0	0000	1101	1011		
0	1011	1101	1011	Add	} First cycle
0	0101	1110	1011	Shift	
0	0010	1111	1011	Shift	} Second cycle
0	1101	1111	1011	Add	
0	0110	1111	1011	Shift	} Third cycle
1	0001	1111	1011	Add	
0	1000	1111	1011	Shift	} Fourth cycle

Note:

M remains the same

C, A & Q involved in shifting

Flowchart for **Unsigned** Binary Multiplication



11 x 13

- 11 = multiplicand
- 13 = multiplier

				1	0	1	1	11
			X	1	1	0	1	13
				1	0	1	1	
			0	0	0	0	x	
		1	0	1	1	x	x	
	1	0	1	1	x	x	x	
1	0	0	0	1	1	1	1	

$$(11)_{10} \times (13)_{10}$$

$$(11)_{10} = (1011)_2$$

$$(13)_{10} = (1101)_2$$

$0 + 0 = 0$
 $0 + 1 = 1$
 $1 + 0 = 1$
 $1 + 1 = 0$ carry forward 1

				1	0	1	1
				1	1	0	1
				1	0	1	1
			1	0	0	0	
		1	0	1	1		
	1	1	0	1	1		
	1	0	1	1			
1	0	0	0	1	1	1	1

{
}

$$(13)_{10} \times (9)_{10}$$

$$(13)_{10} = (1101)_2$$

$$(9)_{10} = (1001)_2$$

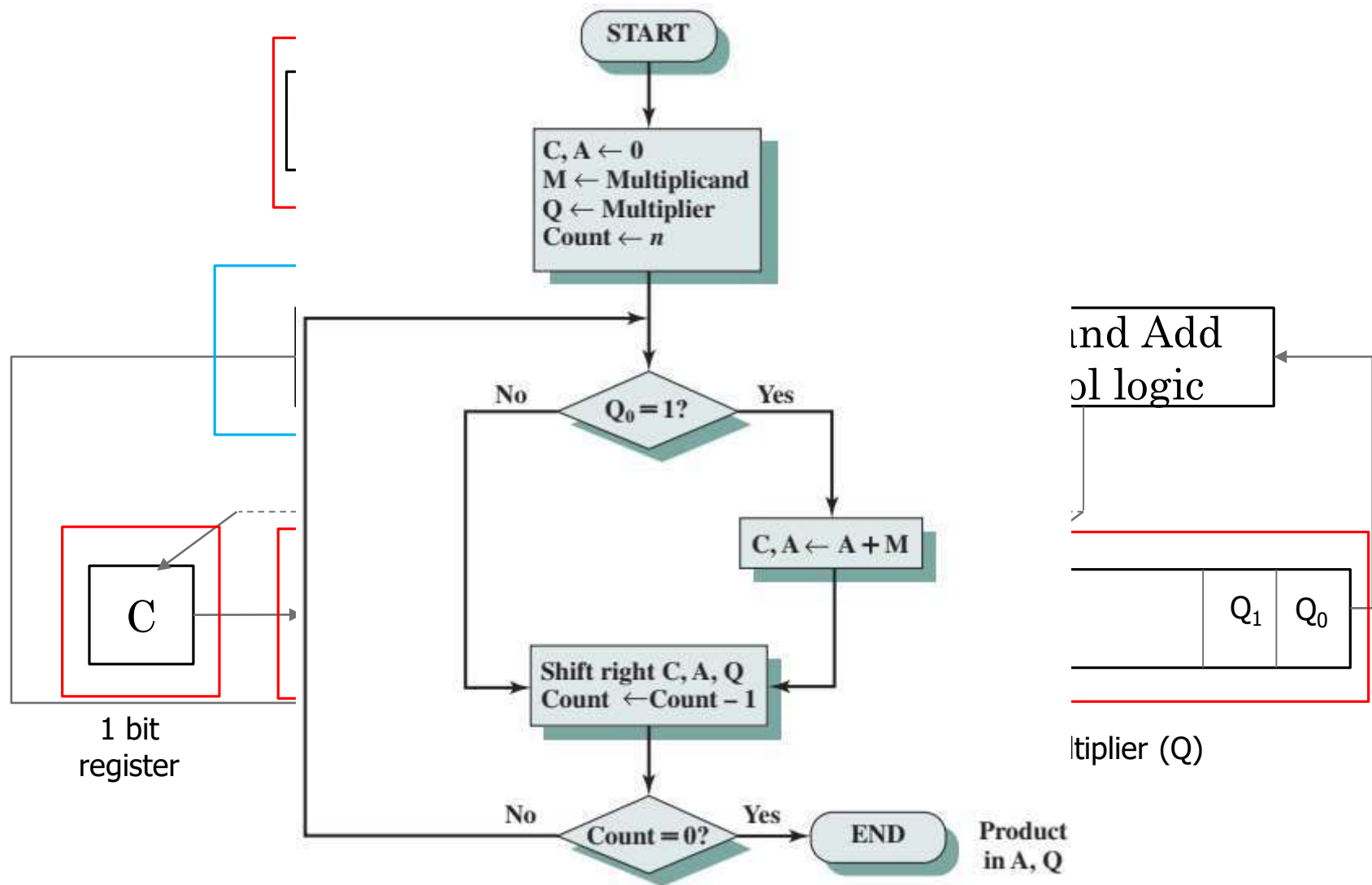
$0 + 0 = 0$
 $0 + 1 = 1$
 $1 + 0 = 1$
 $1 + 1 = 0$ carry forward 1

				1	1	0	1	
				1	0	0	1	
			1	1	1	0	1	
			0	0	0	0		
		0	0	0	0			
	1	1	0	1				
0	1	1	1	0	1	0	1	

$= (117)_{10}$

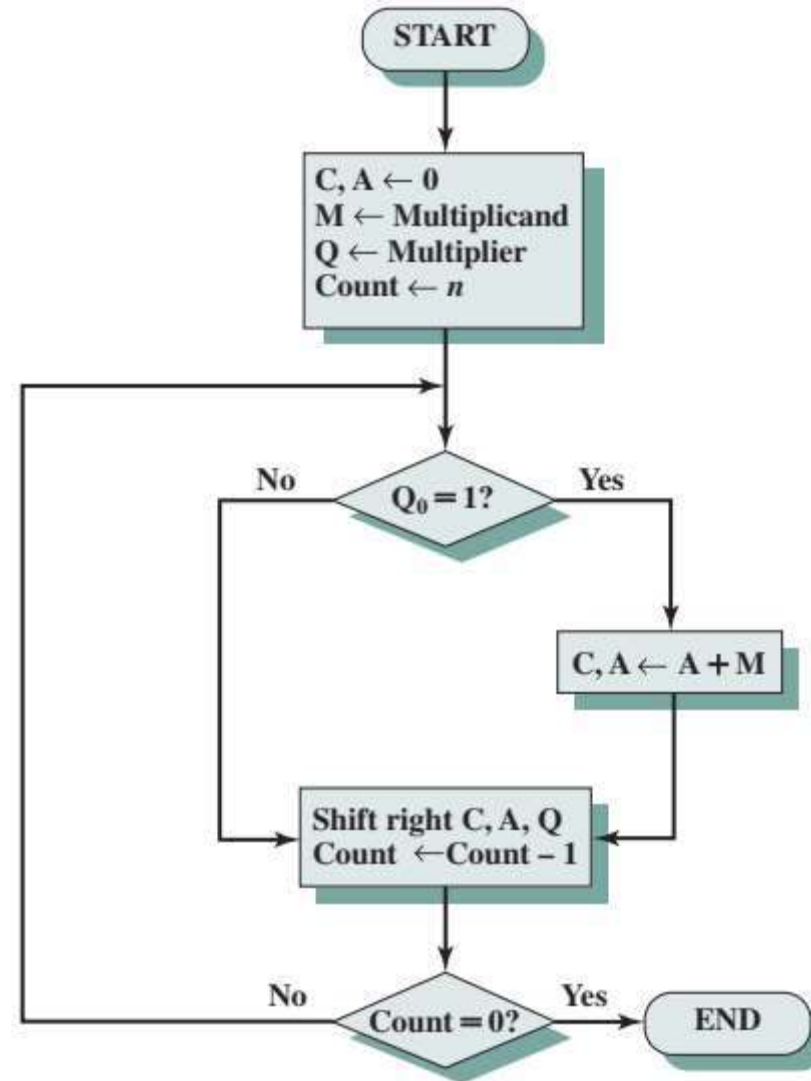
Multiplication using Add-shift method

- 11×13
- $11 = M$ (multiplicand)
- $13 = Q$ (multiplier)
- $143 = AQ$



Hardware Structure for add-shift multiplication

1. Initialize C, A to 0
2. Fill the (M) and (Q) with the binary value
 - Count = bit value
3. N cycle:
 - Check the Q_0 value
 - ✓ If 1 = Add value in (A) and (M)
 - Shift RIGHT (C), (A) and (Q)
 - Count - 1
 - ✓ If 0 = Shift RIGHT (C), (A) and (Q)
 - Count -1
4. Final cycle = count = 0



13 ($M_{\text{multiplicand}}$) = 1101
 11 ($Q_{\text{Multiplier}}$) = 1011

13 x 11
 Expected results:
 $(143)_{10}$
 $(10001111)_2$

C	A	Q	Operation	Cycle
0	0000	1011	Initial	
0	1101	1011	Add (A+M)	1 st
0	0110	1101	Shift right	
1	0110	1101	Add (A+M)	2 nd
0	1001	1110	Shift right	
0	0100	1111	Shift right	3 rd
1	0001	1111	Add (A+M)	4 th
0	1000	1111	Shift right	

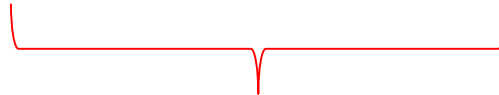
Shift count = bit count
 Add count = 1 count in Q

13 ($M_{\text{multiplicand}}$) = 1101 (remain unchanged)
 11 ($Q_{\text{Multiplier}}$) = 1011

$n=4$

13 x 11
 Expected results:
 $(143)_{10}$
 $(10001111)_2$

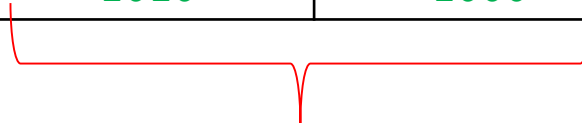
C	A	Q	Operation	Cycle
0	0000	1 0 1 1	initialization	-
0	0000 + 1101 0110 1101	1 0 1 1 ^{Q₃ Q₂ Q₁ Q₀} 1 1 0 1	A+M Shift → C, A & Q	1 st cycle
1	0010 + 1101 1001 11	1 1 0 1 1 1 1 0	A+M Shift → C, A & Q	2 nd cycle
0	0100	1 1 1 1	Shift → C, A & Q	3 rd cycle
1	0000 + 1101 1000 1	1 1 1 1	A+M	4 th cycle
0	10001	1 1 1 1	Shift → C, A & Q	



12 ($M_{\text{multiplicand}}$) = 1100
 14 ($Q_{\text{Multiplier}}$) = 1110

12 x 14
 Expected results:
 $(168)_{10}$
 $(10101000)_2$

C	A	Q	Operation	Cycle
0	0000	1110	Initial	
0	0000	0111	Shift C, A & Q	1 st
0	1100	0111	Add A+M	2 nd
0	0110	0011	Shift C, A & Q	
1	0010	0011	Add A+M	3 rd
0	1001	0001	Shift C, A & Q	
1	0101	0001	Add A+M	4 th
0	1010	1000	Shift C, A & Q	



Quick activity #7

Multiply the following values using add-shift multiplication method:

- 10×10
- 9×12

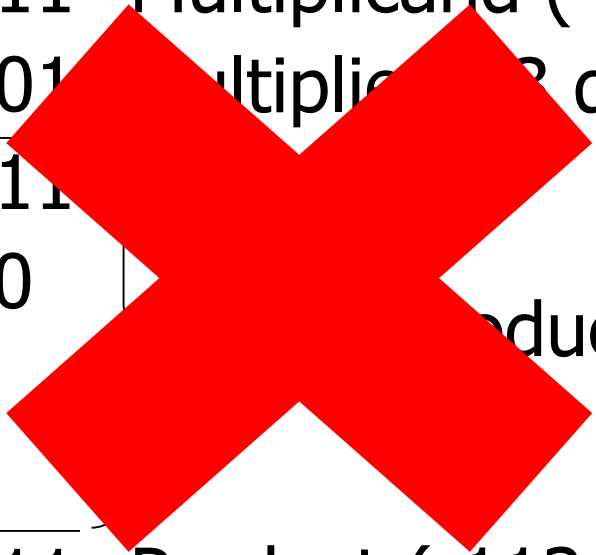
How we handle multiplication of two negative numbers?



Multiplying Negative Numbers (2s Complement)

1011	Multiplicand (-5 decimal)
x 1101	Multiplier (-3 decimal)
1011	} Partial products
0000	
1011	
1011	
10001111	Product (?? decimal)

Multiplying Negative Numbers (2s Complement)


$$\begin{array}{r} 1011 \text{ Multiplicand (-5 decimal)} \\ \times 1101 \text{ Multiplier (-3 decimal)} \\ \hline 1011 \\ 0000 \\ 1011 \\ 1011 \\ \hline 10001111 \text{ Product (-113 decimal)} \end{array}$$

Multiplying Negative Numbers (2s Complement)

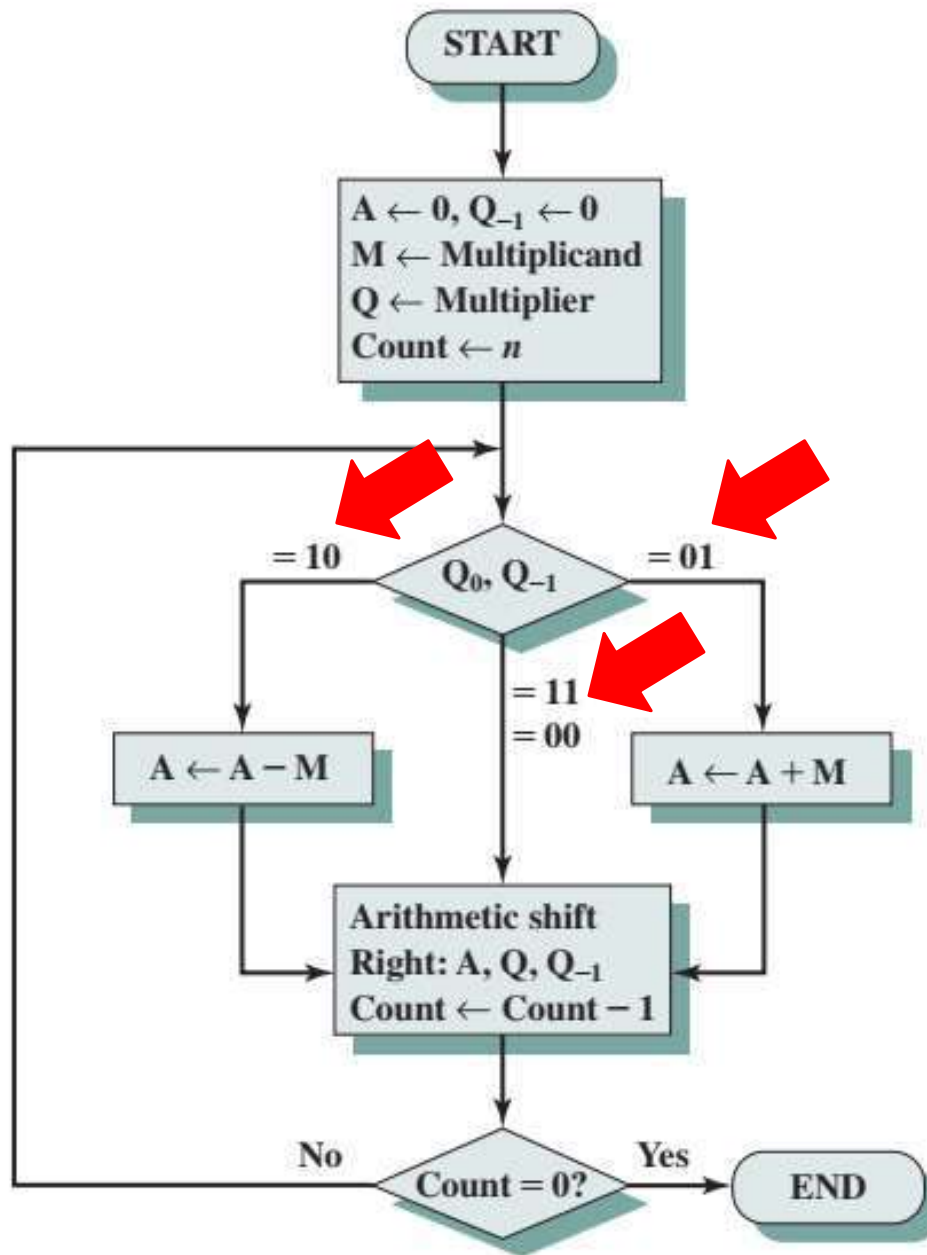
Solution 1

- **Convert to positive** if required
- Multiply as above
- **If signs were different, negate** answer
 - Implementers prefer techniques that do not require this final transformation step

Multi

Solu

- Bo
- M
- co
- In



ient)

bers in 2s

n in 1950s

Booth's Algorithm conditions

$Q_3Q_2Q_1Q_0Q_{-1}$

Q_0	Q_{-1}	Operation
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}

Booth's algorithm

7 x 3

Expected results:

$(21)_{10}$

$(10101)_2$

- 7 ($M_{\text{multiplicand}}$) = 0111(remain unchanged)
- 3 ($Q_{\text{Multiplier}}$) = 0011

A	Q	Q ₋₁	M	Initial values	
0000	0011	0	0111		
1001	0011	0	0111	A ← A - M	First cycle
1100	1001	1	0111	Shift	
1110	0100	1	0111	Shift	Second cycle
0101	0100	1	0111	A ← A + M	Third cycle
0010	1010	0	0111	Shift	
0001	0101	0	0111	Shift	Fourth cycle

Booth's algorithm

- 7 ($M_{\text{multiplicand}}$) = 0111 (remain unchanged)
- 3 ($Q_{\text{Multiplier}}$) = 0011

Q_0	Q_{-1}	Operation
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}

A	Q	Q_{-1}	M	Initial values	
0000	0011	0	0111		
1001	0011	0	0111	$A \leftarrow A - M$	First
1100	1001	1	0111	Shift	cycle
1110	0100	1	0111	Shift	Second cycle
0101	0100	1	0111	$A \leftarrow A + M$	Third cycle
0010	1010	0	0111	Shift	
0001	0101	0	0111	Shift	Fourth cycle

Booth's Algorithm

Example 2

$$7 (M_{\text{multiplicand}}) = 0111$$

$$-6 (Q_{\text{Multiplier}}) = 1010$$

$$2s M = 1001$$

Q_0	Q_{-1}	Operation
0	0	Shift right A, Q, Q_{-1}
1	1	
0	1	Add (A+M) Shift right A, Q, Q_{-1}
1	0	Subtract (A-M) Shift right A, Q, Q_{-1}

A	Q	Q_{-1}	M	Operation	Cycle
0000	1010	0	0111	Initialize	
0000	0101	0	0111	Shift	1 st cycle
1001	0101	0	0111	A-M = A+2sM	2 nd cycle
1100	1010	1	0111	Shift	
0011	1010	1	0111	A+M	3 rd cycle
0001	1101	0	0111	Shift	
1010	1101	0	0111	A-M = A+2sM	4 th cycle
1101	0110	1	0111	Shift	

Booth's Algorithm

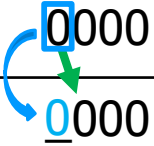
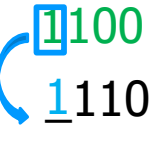
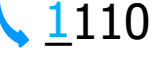
Example 3

Case 1: 4 x 4

$$M = 4 = 0100$$

$$Q = 4 = 0100$$

$$2s\ M = 1100$$

A	Q	Q ₋₁	Operation	Cycle
 0000	0100	0	Init.	
 0000	0010	0	Shift	
0000	0001	0	Shift	
 1100	0001	0	A-M	
 1110	0000	1	Shift	
0010	0000	1	A+M	
0001	0000	0	Shift	

0000
1100
1100

1110
0100
10010

Case 1: 4 x -4

$$M = 4 = 0100$$

$$2s\ M = 1100$$

$$Q = -4 = 1100$$

$$2s\ Q = 0100$$

A	Q	Q ₋₁	Operation	Cycle
0000	1100	0	Init.	
0000	0110	0	Shift	
0000	0011	0	Shift	
1100	0011	0	A-M	
1110	0001	1	Shift	
1111	0000	1	Shift	

0000
1100
1100

Case 1: -4 x 4

$$M = -4 = 1100 \quad 2s \ M = 0100$$

$$Q = 4 = 0100$$

A	Q	Q ₋₁	Operation	Cycle
0000	0100	0	Init.	
0000	0010	0	Shift	
0000	0001	0	Shift	
0100	0001	0	A-M	
0010	0000	1	Shift	
1110	0000	1	A+M	
1111	0000	0	Shift	

0000
 0100
 0100
 0010
 1100
 1110

Case 1: -4 x -4

$$M = -4 = 1100 \quad 2s \ M = 0100$$

$$Q = -4 = 1100 \quad 2s \ Q = 0100$$

A	Q	Q ₋₁	Operation	Cycle
0000	1100	0	Init.	
0000	0110	0	Shift	
0000	0011	0	Shift	
0100	0011	0	A-M	
0010	0001	1	Shift	
0001	0000	1	Shift	

0000
0100
0100

Advantages of Booth's Algorithm

- Performed signed multiplications
- Performs better if –ve numbers are involved
- Reduced number of operations

Quick activity #8

1. Use the Booth algorithm to multiply 23 (multiplicand) by 29 (multiplier), where each number is represented using 6 bits
2. Use the Booth algorithm to multiply 10 (multiplicand) by -10 (multiplier), where each number is represented using 5 bits
3. Use the Booth algorithm to multiply -6 (multiplicand) by -5 (multiplier), where each number is represented using 4 bits
4. Use the Booth algorithm to multiply -7 (multiplicand) by 5 (multiplier), where each number is represented using 4 bits
5. Given $x = 0100$ and $y = 1100$ in two's complement notation (i.e., $x = 4$, $y = -4$), compute the product $p = x * y$ with Booth's algorithm

2's complement operations

Addition

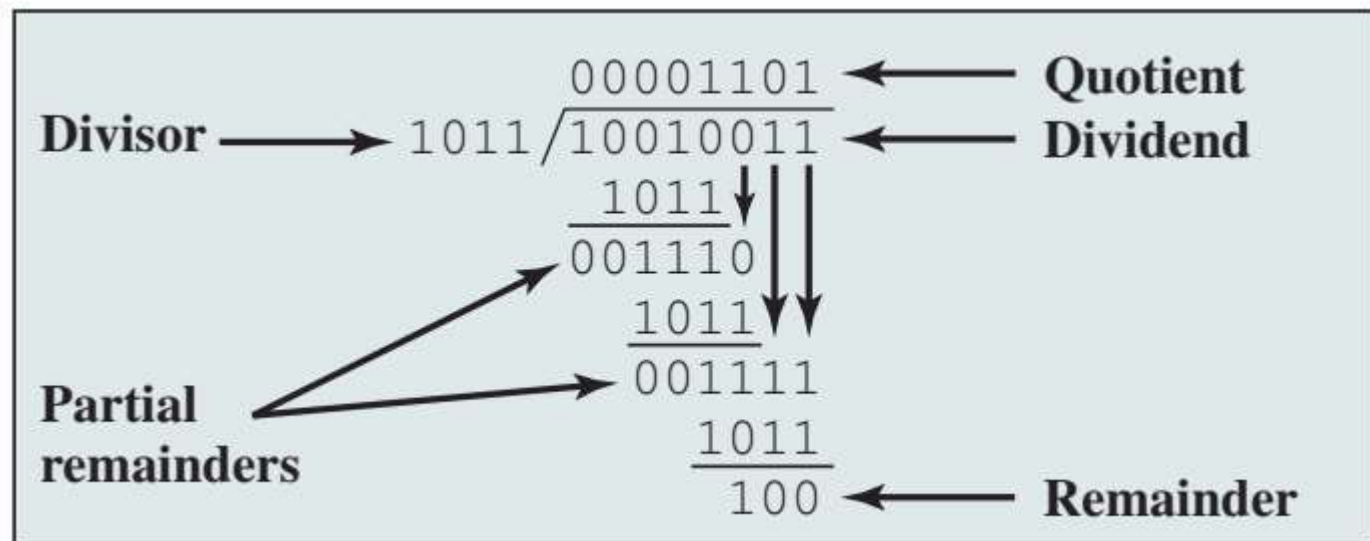
Subtraction

Multiplication

Division

Division

- Division is somewhat more complex than multiplication but is based on the same general principles.
- As before, the basis for the algorithm is the paper-and-pencil approach, and the operation involves repetitive shifting and addition or subtraction.



Long method division

$$\begin{array}{r} 0 \quad 1 \quad 6 \\ 8 \overline{) 1 \quad 2 \quad 8} \\ - 0 \\ \hline 1 \quad 2 \\ 8 \\ \hline 4 \quad 8 \\ 8 \\ \hline 0 \end{array}$$

$$1 < 8$$

$$12 > 8$$

$$48 > 8$$

Dividend (M) = 128

Divisor (Q) = 8

Quotient (or Answer) = 16

Remainder = 0

Long method division

Dividend (M) = 101010 $(42)_{10}$

Divisor (Q) = 110 $(6)_{10}$

$$\begin{array}{r}
 110 \overline{) 101010} \\
 \underline{0} \\
 10 \\
 \underline{0} \\
 101 \\
 \underline{110} \\
 1010 \\
 \underline{1100} \\
 1001 \\
 \underline{1100} \\
 0110 \\
 \underline{1100} \\
 0010 \\
 \underline{0010} \\
 0000
 \end{array}$$

$1 < 110$

$10 < 110$

$101 < 110$

$1010 > 110$

$1001 > 110$

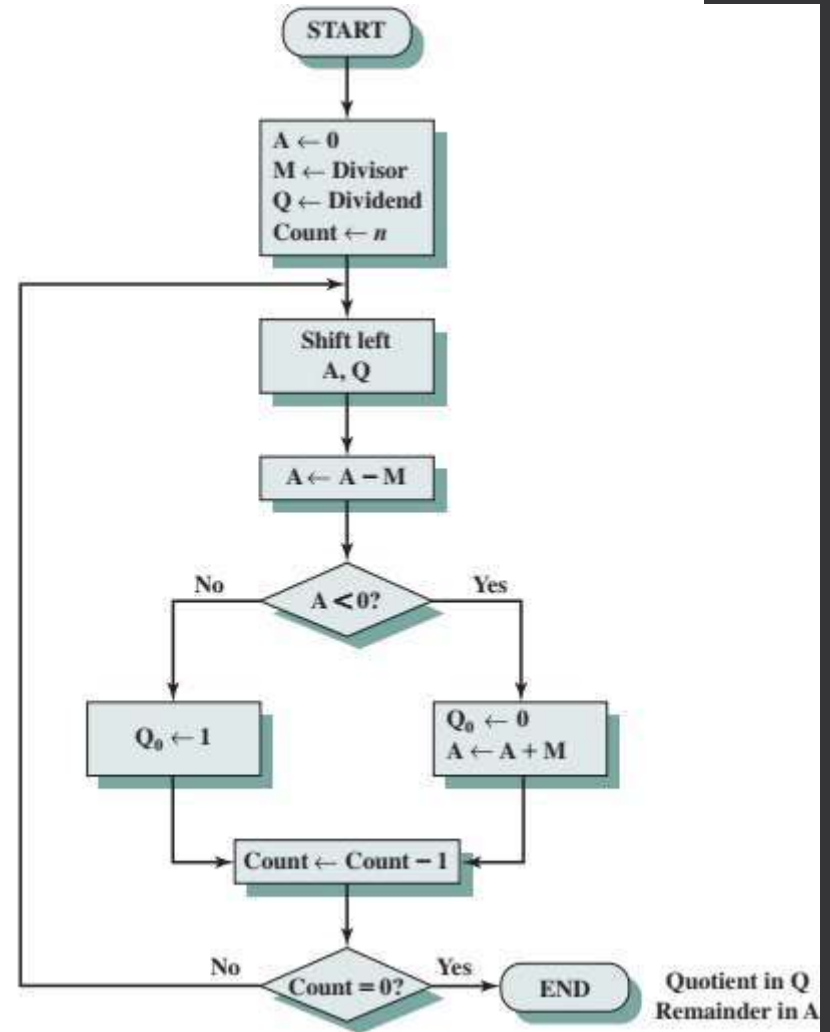
$110 < 110$

Quotient (or Answer) = 111

Remainder = 0

Flowchart for Unsigned Binary Division

1. Initial A register, Load M&Q register
2. Shift left (A & Q)
3. A - M operation
4. Look at A condition:
 - $A < 0 = \text{True} = Q_0$ gets a 1 bit
 - $A < 0 = \text{False} = Q_0$ gets a 0 bit and restores M the previous value



Division in 2s complement

$7 \div 3 = 2$ with 1 remainder ($Q \div M$)

- $Q = 0111$

- $M = 0011$

$$\begin{array}{r} (3) \quad 0011 \overline{) 0111} \quad (7) \\ \underline{011} \\ 0001 \\ \underline{0000} \\ 0001 \end{array}$$

$$2sM = 1101$$

	A 0000	Q 0111	Initial value
(-3)	0000	1110	Shift
	<u>1101</u>		Use twos complement of 0011 for subtraction
	1101		Subtract
	0000	1110	Restore, set $Q_0 = 0$
(-1)	0001	1100	Shift
	<u>1101</u>		
	1110		Subtract
	0001	1100	Restore, set $Q_0 = 0$
(0)	0011	1000	Shift
	<u>1101</u>		
	0000	1001	Subtract, set $Q_0 = 1$
(-1)	0001	0010	Shift
	<u>1101</u>		
	1110		Subtract
	0001	0010	Restore, set $Q_0 = 0$

Q (Dividend) = 7 = 0111
M (Divisor) = 3 = 0011
2sM = 1101

	A	Q	Operation	
	0000	0111	Init.	
(-3)	0000 1101	1110	Shift (L) A-M	0000 1101
	0000	1110	Restore A, $Q_0 = 0$	1101 1101
(-1)	0001 1110	1100	Shift (L) A-M	0001 1101
	0001	1100	Restore A, $Q_0 = 0$	1110 1110
(0)	0011 0000	1000	Shift (L) A-M	0011 1101
	0000	1001	Subtract, $Q_0 = 1$	10000
(-1)	0001 1110	0010	Shift (L) A-M	0001 1101
	0001	0010	Restore A, $Q_0 = 0$	1110

$$Q = 4 = 0100$$

$$M = 2 = 0010 \quad (25 = 1110)$$

	A	Q	op.
	0000	0100	init. value
(-2)	0000	1000	shift L
	1110		A-M
	0000	1000	Restore, $Q_0 = 0$
(-1)	0001	0000	shift L
	1111		A-M
	0001	0000	Restore, $Q_0 = 0$
(0)	0010	0000	shift L
	0000		A-M
	0000	0001	subtract, $Q_0 = 1$
(-2)	0000	0010	shift L
	1110		A-M
	0000	0010	Restore, $Q_0 = 0$
	<u>0000</u>	<u>0010</u>	
	Remainder	Answer	

Quick activity #9

Execute the division for the following binary numbers using the long division method and restoring 2s complement division. You must show the full complete process side by side.

1. $4 \div 2$

2. $9 \div 3$

3. $10 \div 4$

4. $11 \div 3$

Outline

1 Arithmetic & Logic Unit

2 Integer Representation

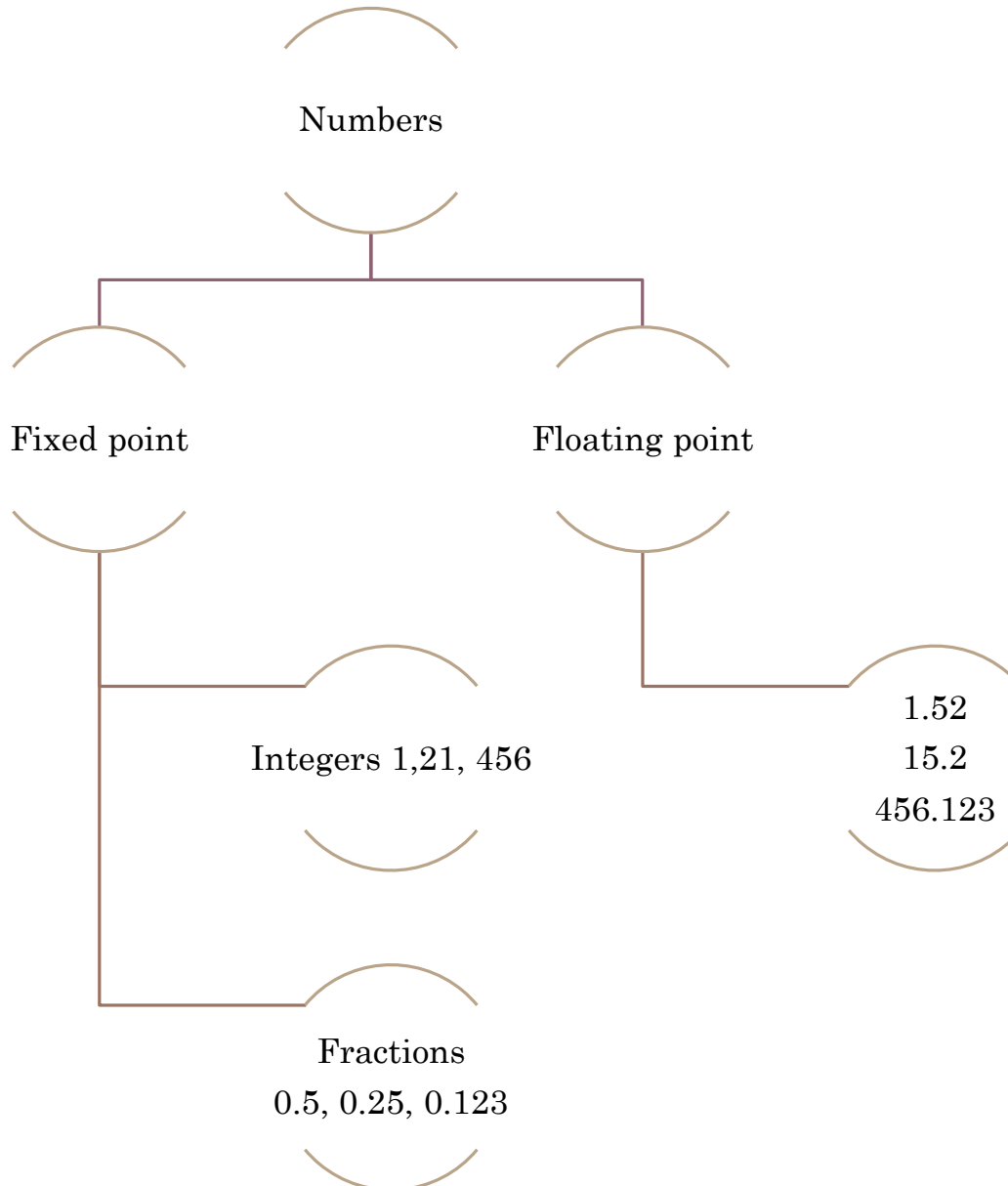
- Sign-magnitude
- Twos complement
- Conversion between lengths

3 Integer Arithmetic

- Addition & Subtraction
- Multiplication
- Division

4 Floating-point Arithmetic

Introduction



Real numbers

- Numbers with fractions
- Could be done in pure binary
- E.g.:

$$1001.1010 = 2^3 + 2^0 + 2^{-1} + 2^{-3} = 9.625$$

Real numbers

- With a fixed-point notation (e.g., twos complement) it is possible to represent a range of positive and negative integers centered on or near 0.
- **HOWEVER**, this approach has limitations
- Cannot represent a very **LARGE** numbers nor very **SMALL** fractions.
- For decimals, we overcome this limitation by using scientific notation

976,000,000,000,000

9.76×10^{14}

0.000000000000000976

9.76×10^{-14}

Real numbers

- Where is the binary or radix point?
- Fixed?
 - Very limited
- Moving?
 - How do we show where it is?

Floating-point representation

- What about binary numbers?

$$\pm S \times B^{\pm E}$$

- Sign: plus, or minus
- Significand S
- Exponent E

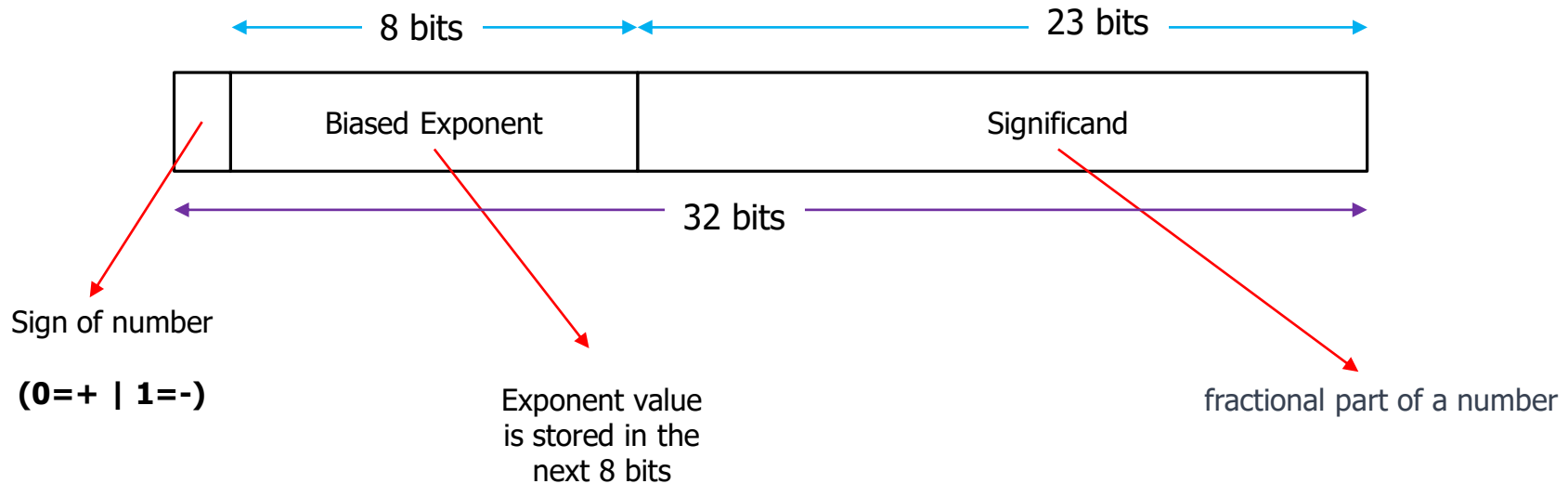


Signs for Floating Point

- Significand is stored in 2s complement
- Exponent is in **biased** notation
 - e.g. Bias of 127 means $(2^{k-1} - 1)$, k is number of bits
 1. **8-bit** exponent field
 1. Pure value range 0-255
 2. Range -127 to +128
 2. Subtract 127 to get the correct value



Floating-point representation



$$1.6328125 \times 2^{20} = 1.101001 \times 2^{10100}$$

Floating-point representation

$$1.6328125 \times 2^{20} = 1.101001 \times 2^{10100}$$

1. The original value of exponent = 20
2. To get the exponent value in binary = original exponent + bias exponent

$$20 + 127 = 147 \rightarrow 10010011$$

3. The biased exponent value is = 10010011



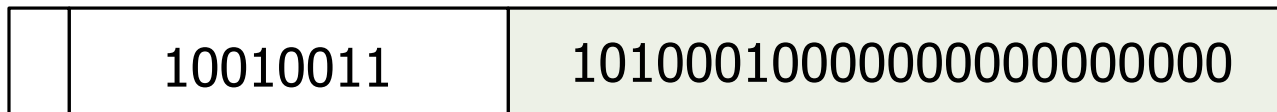
S

Biased Exponent

Significand

Floating-point representation

$$1.6328125 \times 2^{20} = 1.1010001 \times 2^{10100}$$



S

Biased Exponent

Significand

How do we get the significand value?

2. Convert the fractional portion to binary

0.6328125 to binary = 101001

3. Therefore, the significand is 101001 + remaining 0 to fulfil 23 bits

Floating-point representation

$$1.6328125 \times 2^{20} = 1.1010001 \times 2^{10100}$$

0	10010011	101000100000000000000000
---	----------	--------------------------

S

Biased Exponent

Significand

How do we identify the sign value?

0 = (+) and 1 (-)

$$1.1010001 \times 2^{10100} = 0 \ 10010011 \ 101000100000000000000000 = 1.6328125 \times 2^{20}$$

Floating-point representation



Tip: $20 + 127 = 147$

$$\begin{array}{rclcl}
 1.1010001 \times 2^{10100} & = & 0 & \boxed{10010011} & 101000100000000000000000 & = & 1.6328125 \times 2^{20} \\
 -1.1010001 \times 2^{10100} & = & 1 & \boxed{10010011} & 101000100000000000000000 & = & -1.6328125 \times 2^{20} \\
 1.1010001 \times 2^{-10100} & = & 0 & \boxed{01101011} & 101000100000000000000000 & = & 1.6328125 \times 2^{-20} \\
 -1.1010001 \times 2^{-10100} & = & 1 & \boxed{01101011} & 101000100000000000000000 & = & -1.6328125 \times 2^{-20}
 \end{array}$$

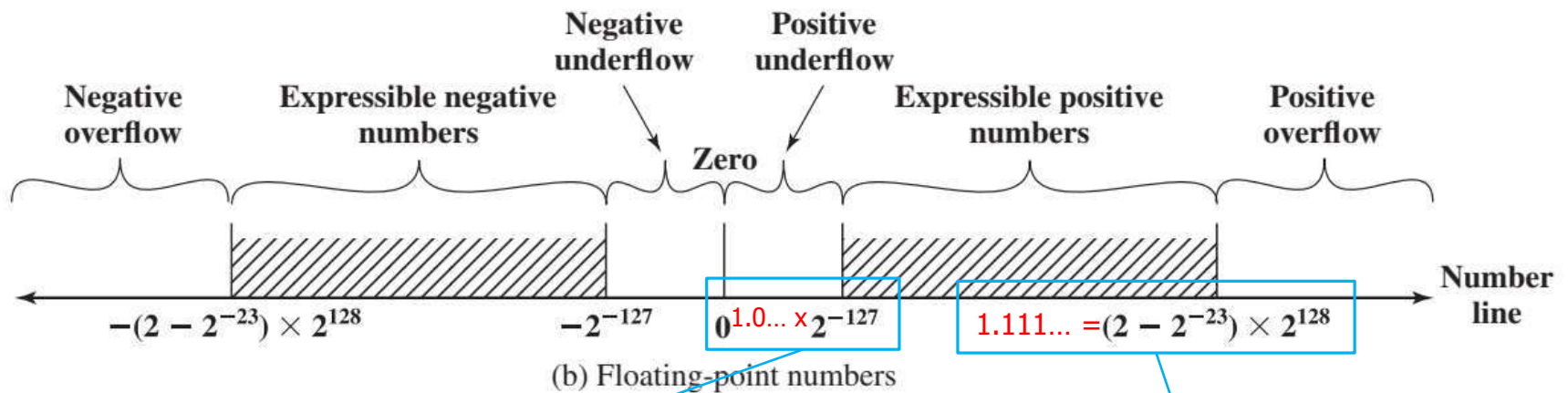
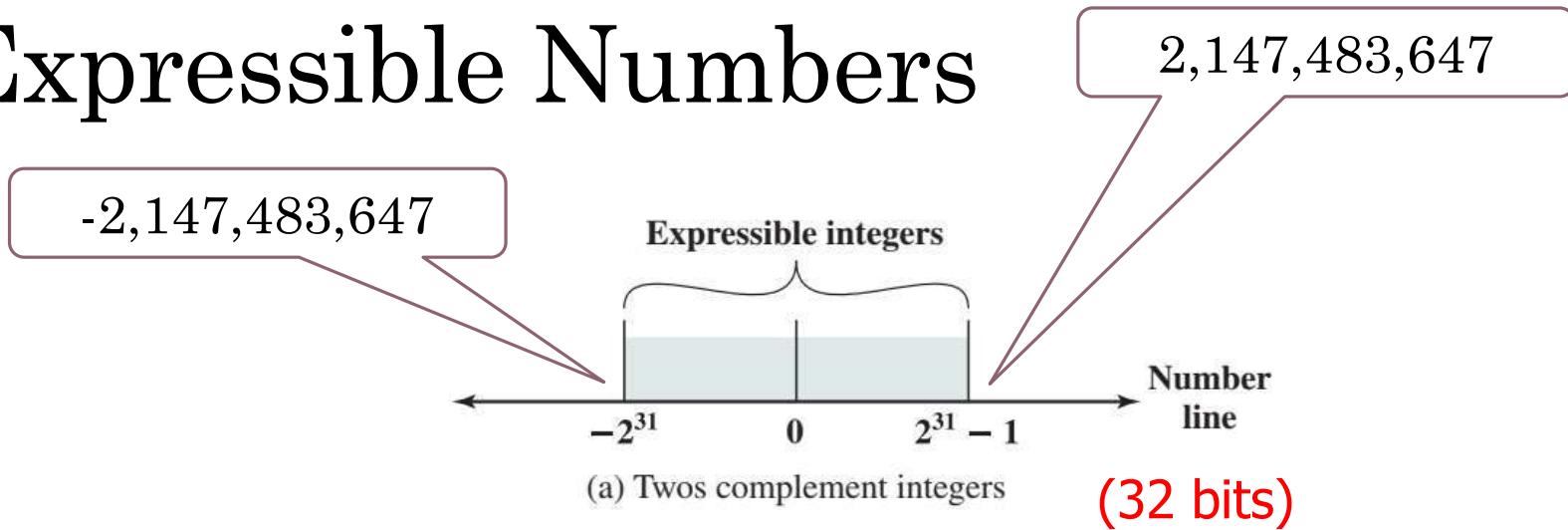
Tip: $-20 + 127 = 107$

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1

Normalization

- Floating point numbers are usually normalized
 - i.e. exponent is adjusted so that **leading bit (MSB) of significand is 1**
- Since it is always 1 there is **no need to store it**
 - 23 bits can store 24-bit number
- c.f. Scientific notation where numbers are normalized to give a single digit before the decimal point
 - e.g. 3.123×10^3

Expressible Numbers



The smallest (+) number can be represented

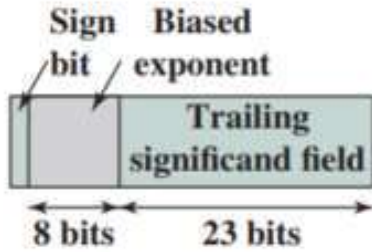
The biggest (+) number can be represented

(1 sign bit, 8 bit exponent, 23-bit significand)

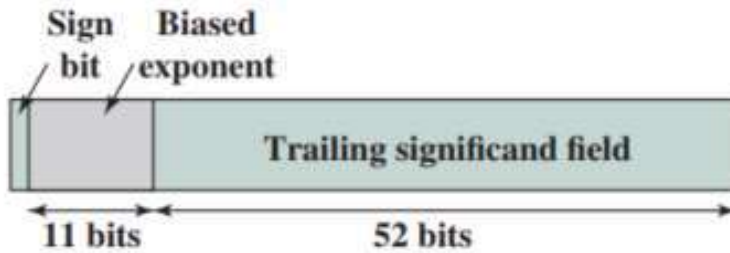
IEEE 754

- The most important floating-point representation is defined in IEEE Standard 754, adopted in 1985 and revised in 2008.
- This standard was developed to facilitate the portability of programs from one processor to another and to encourage the development of sophisticated, numerically oriented programs.
- The standard has been widely adopted and is used on virtually all contemporary processors and arithmetic coprocessors. IEEE 754-2008 covers both binary and decimal floating-point representations.
- Standard for floating point storage
- **32- 64- and 128-bit standards**
- **8, 11 and 15-bit exponent respectively**
- Extended formats (both significand and exponent) for intermediate results

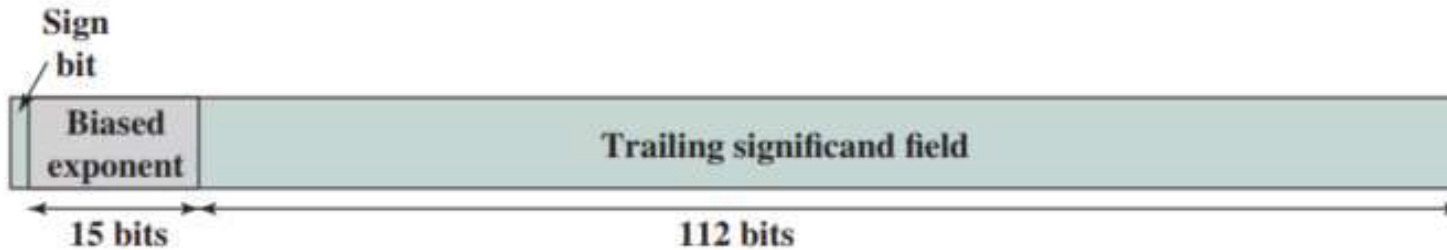
IEEE 754



(a) Binary32 format



(b) Binary64 format



(c) Binary128 format

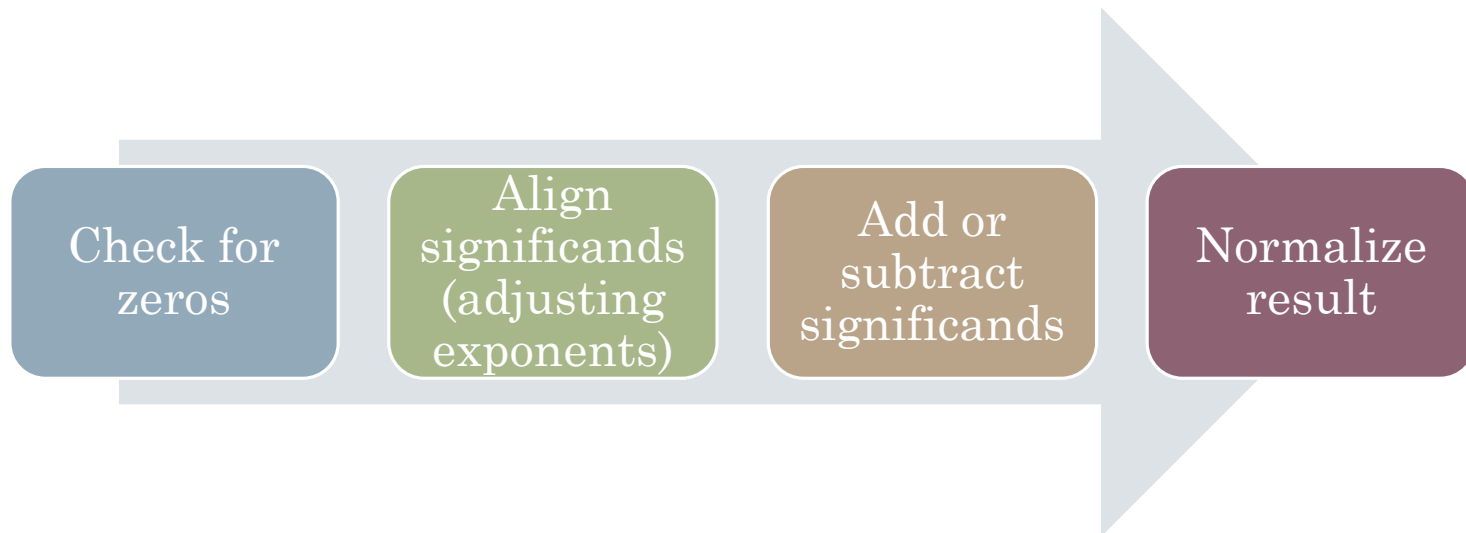
Parameter	Format		
	Binary32	Binary64	Binary128
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Exponent bias	127	1023	16383
Maximum exponent	127	1023	16383
Minimum exponent	-126	-1022	-16382
Approx normal number range (base 10)	$10^{-38}, 10^{+38}$	$10^{-308}, 10^{+308}$	$10^{-4932}, 10^{+4932}$
Trailing significand width (bits)*	23	52	112
Number of exponents	254	2046	32766
Number of fractions	2^{23}	2^{52}	2^{112}
Number of values	1.98×2^{31}	1.99×2^{63}	1.99×2^{128}
Smallest positive normal number	2^{-126}	2^{-1022}	2^{-16362}
Largest positive normal number	$2^{128} - 2^{104}$	$2^{1024} - 2^{971}$	$2^{16384} - 2^{16271}$
Smallest subnormal magnitude	2^{-149}	2^{-1074}	2^{-16494}

Floating Point Arithmetic

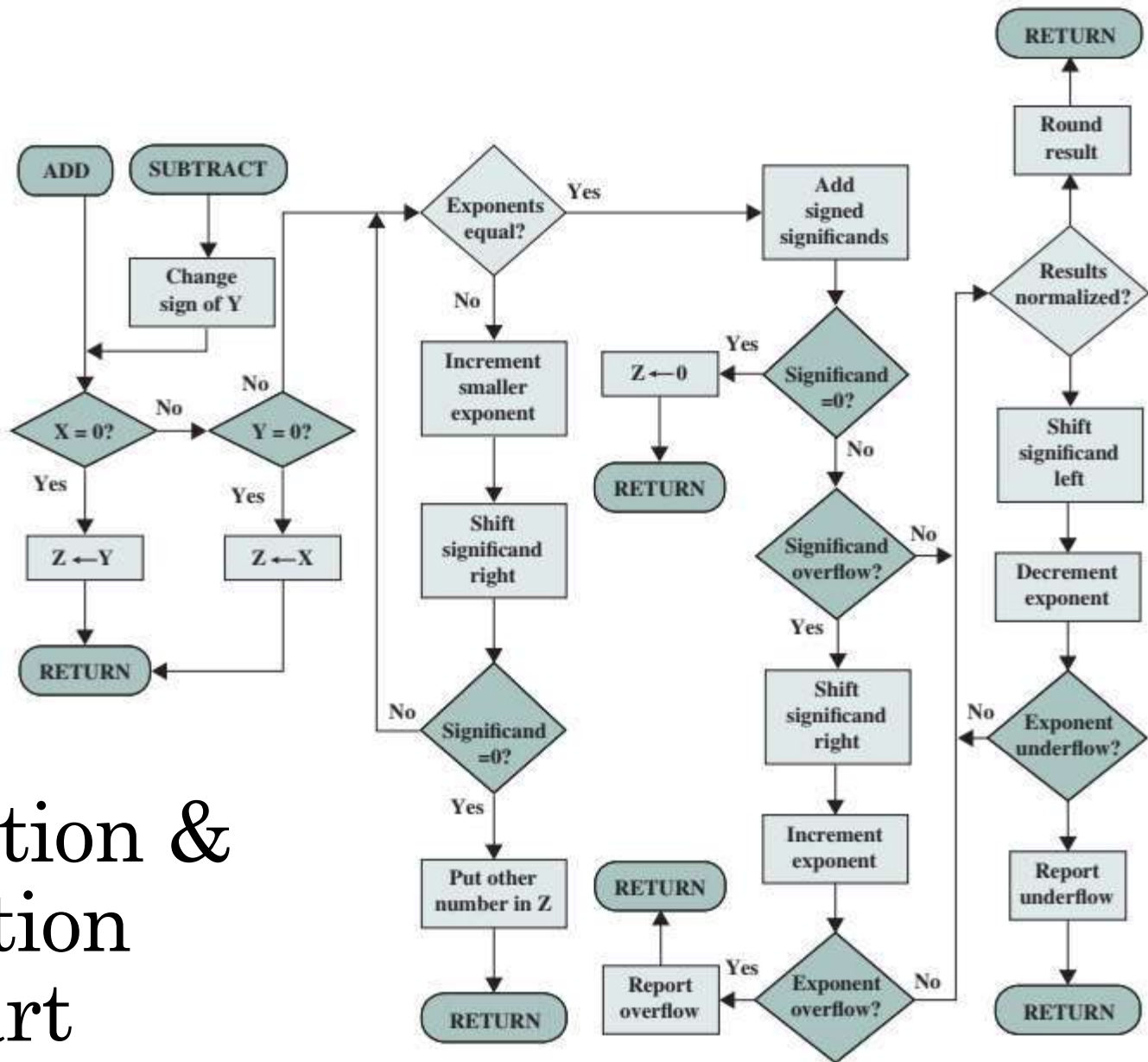
- FP operation may cause
 - Exponent overflow
 - Positive exponent exceed maximum possible value
 - May be $+\infty$ or $-\infty$
 - Exponent underflow
 - Negative exponent less than the minimum possible
 - Number too small, may be reported as 0
 - Significand underflow
 - When aligning, digits flow off the right end of significand
 - Significand overflow
 - Carry out of most significant bit
 - E.g. when adding 2 significand of same sign

FP Arithmetic Addition/Subtraction

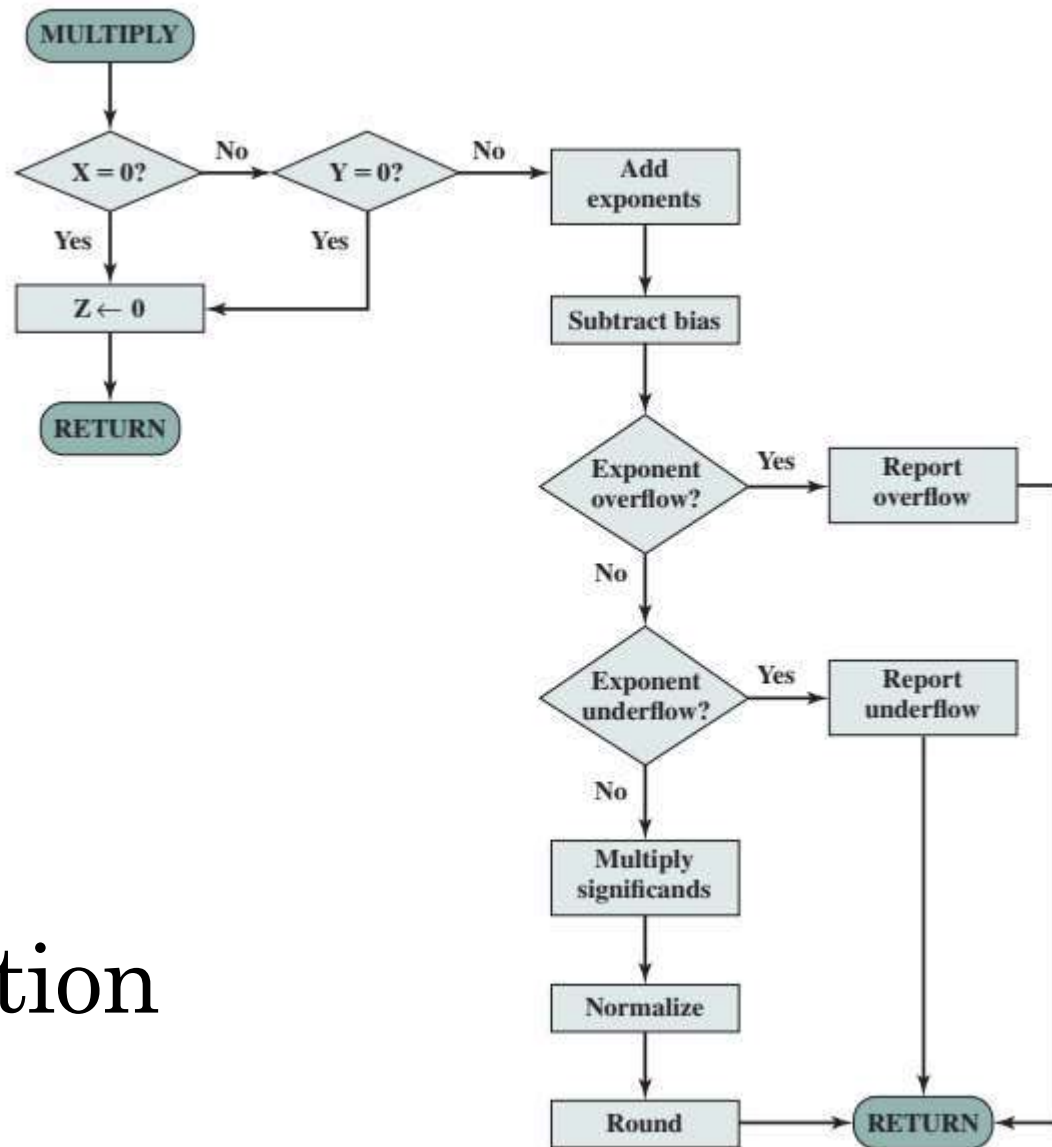
- More complicated than multiplication/division
 - Need for alignment
- 4 phases



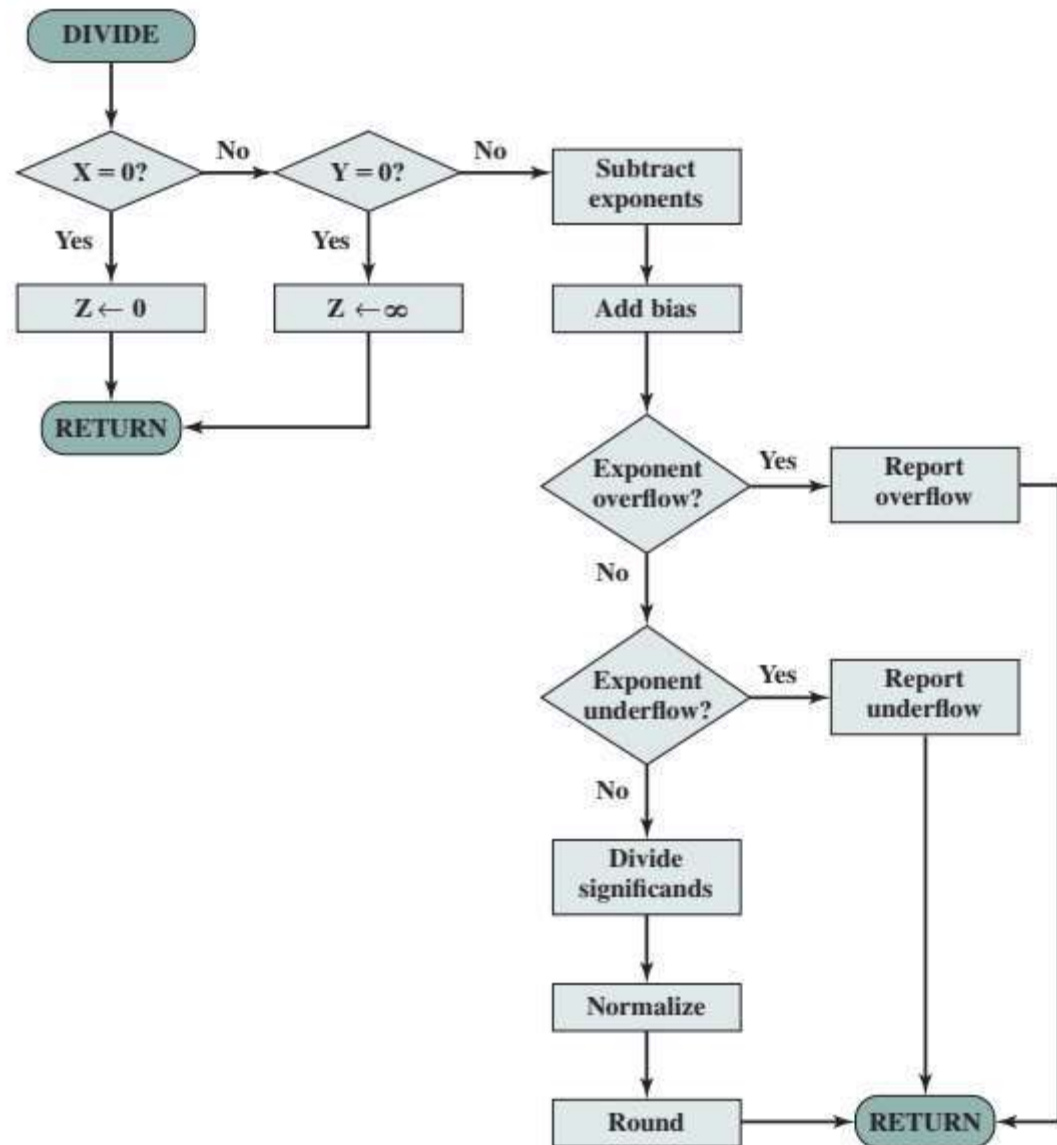
FP Addition & Subtraction Flowchart



FP Multiplication Flowchart



FP Division Flowchart



Quick Activity #10

The following numbers use the IEEE 32-bit floating-point format. What is the equivalent decimal value?

- 0 10000000 101000000000000000000000
- 1 10000010 001000000000000000000000

Quick Activity #11

Express the following numbers in IEEE 32-bit floating-point format

- -5
- -6
- 384
- $1/16$

Quick Activity #11

Express the following numbers in IEEE 32-bit floating-point format

• -5

1	01111111	010000000000000000000000
---	----------	--------------------------

S

Biased Exponent

Significand

1. Determine the sign bit
2. Convert the integer to binary value
3. Determine the exponent
 1. For 32-bit floating point, we will be using 8-bit biased exponent
4. Determine the significand

Reflection...

A person is sitting on a grassy bank next to a body of water. A bicycle is parked on the grass to the right. In the background, there are trees and a building. The scene is captured in a low-angle, slightly blurred style, emphasizing the foreground grass.

From the lecture today, I
learnt.....
Link on eLearn / Chat